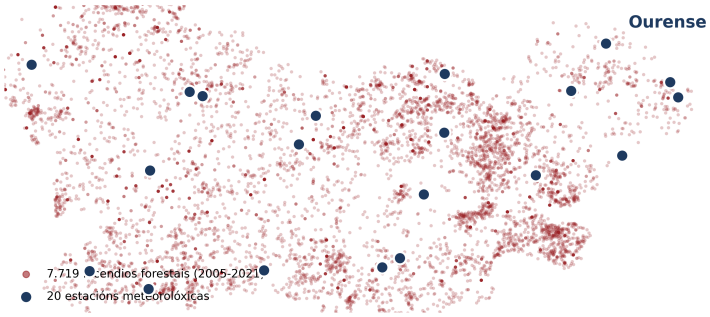
Predición de incendios forestais en Ourense mediante aprendizaxe automática



Lúa Piñeiro Cabanelas

Dimitrii Rapoport Shakin

Mikel Brea Gómez

Traballo realizado no bacharelato **STEMbach**

Bienio 2024–2026

Baixo a dirección conxunta de:

Prof. José Ángel Cid Araújo

Departamento de Matemáticas

Universidade de Vigo

Prof. Carlos Ferreiro García

IES As Lagoas de Ourense

Aplicación web pública: lumia.cabanelas.info

Ourense, maio de 2026

Agradecementos

A **Carlos Ferreiro García**, profesor de Matemáticas e do programa STEMbach no IES As Lagoas, polo seu acompañamento ao longo de todo o bienio e por orientarnos na bibliografía científica que fundamenta as decisións metodolóxicas deste traballo.

A **José Ángel Cid Araújo**, do Departamento de Matemáticas da Universidade de Vigo, pola titoría universitaria e o seguimento académico.

A **MeteoGalicia** e á **Fundación Civio** pola súa política de datos abertos: sen ela, non sería posible construír o dataset que sustenta o noso proxecto.

Resumo

Os incendios forestais son un dos principais problemas medioambientais de Galicia. Ourense é a provincia máis afectada de toda España. Este traballo predí as condicións meteorolóxicas favorables a incendio forestal mediante aprendizaxe automática (*machine learning*).

Para iso construímos un dataset propio cruzando dúas fontes públicas: o rexistro histórico de incendios forestais da Fundación Civio (procedente da Estatística Xeral de Incendios Forestais (EGIF) do Ministerio) e os datos meteorolóxicos diarios da API de MeteoGalicia. Seleccionamos 20 estacións meteorolóxicas da provincia de Ourense e cruzámolas espacial e temporalmente cos 7.719 incendios rexistrados entre 2005 e 2021. Tras un filtro estrito de calidade, procesamos 6.528 incendios (84,6 %) e descartamos 1.191 (15,4 %) sen datos meteorolóxicos completos para o seu día. O dataset final contén 93.221 observacións con 22 variables preditoras.

Adestramos dous modelos de clasificación binaria — Logistic Regression como liña base e Random Forest como modelo principal — avaliados con métricas adecuadas a datos desbalanceados (PR-AUC, F1-score, Recall). O Random Forest acadou un PR-AUC de 0,1485, 9 veces superior ao azar e un 68 % mellor que o modelo lineal. A análise de importancia de variables mostra que os indicadores de seca (días sen choiva, precipitación acumulada) e a humidade relativa son os factores meteorolóxicos máis determinantes na predición de incendios. Este patrón está en consonancia coa literatura científica forestal.

Resumen

Los incendios forestales son uno de los principales problemas medioambientales de Galicia. Ourense es la provincia más afectada de toda España. Este trabajo predice las condiciones meteorológicas favorables a incendio forestal mediante aprendizaje automático (*machine learning*).

Para ello construimos un dataset propio cruzando dos fuentes públicas: el registro histórico de incendios forestales de la Fundación Civio (procedente de la Estadística General de Incendios Forestales (EGIF) del Ministerio) y los datos meteorológicos diarios de la API de MeteoGalicia. Seleccionamos 20 estaciones meteorológicas de la provincia de Ourense y las cruzamos espacial y temporalmente con los 7.719 incendios registrados entre 2005 y 2021. Tras un filtro estricto de calidad, procesamos 6.528 incendios (84,6 %) y descartamos 1.191 (15,4 %) sin datos meteorológicos completos para su día. El dataset final contiene 93.221 observaciones con 22 variables predictoras.

Entrenamos dos modelos de clasificación binaria — Logistic Regression como línea base y Random Forest como modelo principal — evaluados con métricas adecuadas a datos desbalanceados (PR-AUC, F1-score, Recall). El Random Forest alcanzó un PR-AUC de 0,1485, 9 veces superior al azar y un 68 % mejor que el modelo lineal. El análisis de importancia de variables muestra que los indicadores de sequía (días sin lluvia, precipitación acumulada) y la humedad relativa son los factores meteorológicos más determinantes en la predicción de incendios, en consonancia con la literatura científica forestal.

Abstract

Forest fires are one of the main environmental challenges in Galicia. Ourense is the most affected province in all of Spain. We address the prediction of meteorological conditions favourable to forest fires using machine learning.

We built a custom dataset by cross-referencing two public data sources: the historical forest fire record from Fundación Civio (based on the official Spanish General Forest Fire Statistics, EGIF, of the Ministry) and daily meteorological data from the MeteoGalicia API. We selected twenty weather stations in the province of Ourense and matched them spatially and temporally with the 7,719 fires recorded between 2005 and 2021. After a strict quality filter, we processed 6,528 fires (84.6 %) and discarded 1,191 (15.4 %) lacking complete meteorological data for their day. The final dataset contains 93,221 observations with 22 predictor variables.

We trained two binary classification models — Logistic Regression as a baseline and Random Forest as the primary model — evaluated using metrics suited for imbalanced data (PR-AUC, F1-score, Recall). The Random Forest achieved a PR-AUC of 0.1485, nine times above chance and 68 % better than the linear baseline. Feature importance shows that drought indicators (consecutive days without rain, accumulated precipitation) and relative humidity are the most determinant meteorological factors for fire prediction, in line with the forestry science literature.

Palabras chave:

- incendios forestais
- predición
- aprendizaxe automática
- Random Forest
- meteoroloxía
- Ourense
- MeteoGalicia

Keywords:

- forest fires
- prediction
- machine learning
- Random Forest
- meteorology
- Ourense
- MeteoGalicia

Índice Xeral

1 Introdución **1**

2 Antecedentes **2**

2.1 Os incendios forestais en Galicia . . . 2

2.2 Sistemas de predición existentes . . . 2

2.3 Aprendizaxe automática aplicada a incendios forestais . . . 2

3 Hipótese e obxectivos **3**

3.1 Hipótese . . . 3

3.2 Obxectivos . . . 3

4 Ferramentas e métodos **4**

4.1 Fontes de datos . . . 4

4.1.1 Datos de incendios: Fundación Civio (EGIF) . . . 4

4.1.2 Datos meteorolóxicos: API de MeteoGalicia . . . 4

4.2 Limpeza e preprocesamento . . . 4

4.3 Fusión espacial-temporal . . . 5

4.4 Construción do dataset de clasificación . . . 5

4.5 Estratexia de modelado . . . 6

5 Resultados **7**

5.1 Descrición do dataset . . . 7

5.2 Modelo de referencia (*baseline*): Logistic Regression . . . 7

5.3 Modelo principal: Random Forest . . . 8

5.4 Comparación de modelos . . . 9

5.5 Importancia de variables . . . 9

6 Discusión **12**
6.1 Validación da hipótese . . . 12
6.2 Limitacións do enfoque . . . 13
6.3 Coherencia coa literatura científica . . . 13

7 Conclusións **14**
7.1 Conclusións do proxecto . . . 14
7.2 Reflexión sobre o proceso de investigación . . . 14
7.3 Difusión: a interface LUMIA . . . 15
7.4 Proposta de continuidade . . . 15

A Detalles metodolóxicos **17**
A.1 Conversión de coordenadas UTM–WGS84 . . . 17
A.2 Estratexia de división temporal vs. aleatoria . . . 17
A.3 Configuración do Random Forest . . . 17
A.4 Métricas de avaliación e PR-AUC . . . 18
A.5 Importancia completa das 22 variables . . . 18
A.6 Algoritmo de construción do dataset (pseudocódigo) . . . 19
A.7 Ferramentas e reproducibilidade . . . 21

B Regresión Loxística e Bosques Aleatorios **23**
B.1 Regresión Loxística . . . 23
B.2 Bosques Aleatorios (*Random Forest*) . . . 26
B.3 Comparación entre os dous modelos . . . 29

C Caderno de campo **30**
C.1 Prólogo . . . 30
C.2 Sobre o uso de intelixencia artificial neste proxecto . . . 30
C.3 Paso 0 · Como nace o proxecto . . . 31
C.4 Paso 1 · As primeiras descargas . . . 32
C.5 Paso 2 · Comprobando o que tiñamos . . . 32
C.6 Paso 3 · Capando o ruído . . . 33
C.7 Paso 4 · Cruzar dous mundos: o problema das coordenadas . . . 34
C.8 Paso 5 · Inventando os «non incendios» . . . 35
C.9 Paso 6 · Que modelos probar . . . 37
C.10 Paso 7 · O modelo de referencia: Logistic Regression . . . 38
C.11 Paso 8 · Random Forest: o salto cualitativo . . . 39
C.12 Paso 9 · Auditoría e correccións de últimos días . . . 40
C.13 Paso 10 · LUMIA: a interface para mostrar o modelo en directo . . . 42

C.14 Paso 11 · A pregunta de Carlos: que predí exactamente o modelo? 43
C.15 Paso 12 · Cerre: o que aprendemos, o que queda 44
C.15.1 Que aprendemos 44
C.15.2 O que queda pendente 45
C.15.3 O que faríamos diferente 45
C.15.4 Agradecementos 45

D Código fonte **47**
D.1 `fusionar_datasets.py` 47
D.2 `construir_dataset_clasificacion.py` 55
D.3 `entrenar_modelos.py` 61
D.4 `entrenar_random_forest.py` 67
D.5 `exportar_modelo.py` 73

Bibliografía **78**

Capítulo 1

Introdución

Galicia é a comunidade autónoma con maior incidencia de incendios forestais de España: entre 1983 e 2017, acumulou case o 49% da superficie queimada intencionadamente no país, e 70 dos 100 concellos con máis incendios están en territorio galego [4]. Dentro de Galicia, Ourense é a provincia máis afectada dende hai décadas, con casos recentes coma o do verán de 2025, que arrasou unha sétima parte do seu territorio: o peor verán de incendios do século XXI na comunidade autónoma [11].

Os danos non son só medioambientais: tamén económicos e sociais. Cada ano, os incendios forestais destrúen miles de hectáreas, provocan perdas económicas e, nos peores casos, poñen en risco vidas humanas. Por iso, a prevención é clave, e depende sobre todo de anticipar cando as condicións meteorolóxicas favorecen a aparición dun incendio.

Para responder a esta necesidade de anticipación, a Xunta de Galicia publica diariamente o Índice de Risco Diario de Incendio (IRDI) [**?**], baseado no sistema canadense **Fire Weather Index** (FWI, índice meteorolóxico de risco de incendio). Este índice combina temperatura, humidade relativa, velocidade do vento e precipitación nun nivel de risco mediante fórmulas empíricas. Aquí presentamos un enfoque complementario: usar aprendizaxe automática (*machine learning*) para aprender dos datos históricos qué combinacións de condicións meteorolóxicas se asocian con incendios.

Concretamente, investigamos se un modelo adestrado con datos reais de Ourense pode identificar patróns de risco, e que variables meteorolóxicas resultan máis determinantes. Para iso, construímos un dataset propio cruzando incendios con datos meteorolóxicos diarios de 20 estacións da provincia entre 2005 e 2021.

Capítulo 2

Antecedentes

2.1 Os incendios forestais en Galicia

Galicia ten un patrón distinto ao resto de España no estacional dos incendios. Mentres que na maior parte da península ibérica os incendios arden sobre todo nos meses de verán (xullo–setembro), en Galicia hai un dobre pico: un estival, asociado ás altas temperaturas e á seca, e outro invernal (febreiro–marzo), vinculado ás queimas agrícolas tradicionais. No noso dataset de 7.719 incendios en Ourense (2005–2021), este patrón confírmase: agosto é o mes con máis incendios (1.482), seguido de marzo (1.264) e febreiro (1.063).

2.2 Sistemas de predición existentes

O sistema de referencia para a predición do risco de incendios é o Fire Weather Index (FWI), desenvolvido polo Servizo Forestal de Canadá nos anos 70. O FWI combina catro variables meteorolóxicas (temperatura, humidade, vento e precipitación) nun índice numérico que estima canta humidade lle queda á vexetación e ao chan, e polo tanto que probabilidade hai de que arda. En Galicia, a Xunta publica o IRDI (Índice de Risco Diario de Incendio), baseado nesta familia de modelos, con mapas diarios de risco durante a época estival. A AEMET (Axencia Estatal de Meteoroloxía) tamén calcula o FWI para todo o territorio nacional.

2.3 Aprendizaxe automática aplicada a incendios forestais

O *machine learning* aplicado a incendios é un campo activo: **(author?)** [3] é unha referencia clásica, con datos meteorolóxicos do nordeste de Portugal e modelos como SVM (máquinas de vectores de soporte) e Random Forest. Estudos máis recentes [5, 6, 9] demostran que os modelos baseados en árbores — Random Forest [1], XGBoost, CatBoost — superan as redes neuronais en datos tabulares de tamaño medio, como os deste proxecto.

Capítulo 3

Hipótese e obxectivos

3.1 Hipótese

As condicións meteorolóxicas diarias (temperatura, humidade relativa, precipitación, vento, irradiación solar e variables derivadas como a acumulación de seca) conteñen información suficiente para distinguir, cun rendemento superior ao azar, entre días con e sen ocorrencia de incendio forestal na provincia de Ourense.

3.2 Obxectivos

1. **Construír un dataset propio** cruzando datos históricos de incendios forestais en Ourense (Fundación Civio/EGIF) con datos meteorolóxicos diarios de MeteoGalicia, cubrindo o período 2005–2021.

2. **Adestrar e avaliar modelos de clasificación** que predigan o risco de incendio a partir de variables meteorolóxicas, comparando un modelo lineal (Logistic Regression) cun modelo non lineal baseado en árbores (Random Forest).

3. **Identificar as variables meteorolóxicas máis relevantes** para a predición de incendios mediante a análise da **importancia das variables** (en inglés *feature importance*).

4. **Avaliar as limitacións** do enfoque e propoñer melloras futuras.

Capítulo 4

Ferramentas e métodos

4.1 Fontes de datos

4.1.1 Datos de incendios: Fundación Civio (EGIF)

Empregamos o dataset público da Fundación Civio [**?**], que recolle os datos oficiais da Estatística Xeral de Incendios Forestais (EGIF) do Ministerio. O ficheiro contén 292.181 incendios rexistrados en España entre 1968 e 2023, incluíndo data, coordenadas xeográficas, concello, superficie queimada e causas.

Centramos a análise na provincia de Ourense por ser a máis afectada de Galicia, e o período 2005–2021, que vén dado pola cobertura completa de MeteoGalicia desde 2005 e pola última actualización do dataset de Civio (2021).

Tras filtrar pola provincia de Ourense e o período 2005–2021 (solapamento cos datos meteorolóxicos), obtivemos 7.719 incendios, o 99,99% con coordenadas xeográficas precisas.

4.1.2 Datos meteorolóxicos: API de MeteoGalicia

Descargamos datos diarios das **20 estacións máis antigas de MeteoGalicia** en Ourense (operativas desde 2005–2007) vía **API pública** (un servizo automatizado de descarga de datos) [**?**]; descartamos as 14 restantes por dispor só de datos desde 2015–2017.

Por estación obtivemos 12 variables: temperaturas (media/máx./mín.), humidade (media/mín.), precipitación, vento (medio/refacho), irradiación, horas de sol, orballo e presión. Total: 106.928 rexistros diarios (286 MB JSON).

4.2 Limpeza e preprocesamento

A limpeza de incendios consistiu en filtrar por provincia/período e eliminar un único rexistro sen coordenadas. Na meteorolóxica aplicamos filtrado por códigos de validación de

MeteoGalicia (válidos) e corrección de valores atípicos mediante rangos físicos (só 8 atípicos en máis dun millón de medidas). Para ocos illados ($\leq$2 días) usamos **interpolación lineal** (rechear o día faltante coa media dos dous días adxacentes con dato). Catro estacións sen anemómetro (Entrimo, Monte Medo, As Petarelas, Ourense) herdan o vento da estación con anemómetro máis próxima.

4.3 Fusión espacial-temporal

A fusión tivo un problema de coordenadas: os incendios veñen en latitude/lonxitude (WGS84) e as estacións en UTM zona 29N. Implementamos a conversión exacta para calcular distancias en metros (detalles no anexo A).

Para cada incendio buscamos a estación máis próxima cunha estratexia de respaldo (segunda, terceira...) que dispuxese das tres variables base ese día (temperatura, humidade e precipitación). O criterio é estrito: **descartamos 1.191 incendios (15,4 %)** sen datos completos, polo que os **6.528 procesados (84,6 %)** teñen meteoroloxía real do día, sen valores imputados. Distancia mediana á estación asignada: **10,1 km**.

4.4 Construción do dataset de clasificación

Definición formal de positivo, negativo e excluído

Sexa E o conxunto das 20 estacións meteorolóxicas e D o conxunto de días do período 2005–2021. Cada fila do dataset corresponde a unha combinación dun día $d \in D$ e unha estación $e \in E$, etiquetada da seguinte maneira:

- **Positivo** ($y = 1$): par (d, e) tal que existe un incendio rexistrado na estación e na data d (entendendo *na estación* como "asignada a e no paso de fusión espacial-temporal", segundo o criterio de proximidade descrito antes).
- **Excluído** (descartado, non entra no dataset): par (d, e) sen incendio, pero tal que $|d - d_i| \leq 3$ días para algún incendio d_i na mesma estación e.
- **Negativo** ($y = 0$): par (d, e) sen incendio, e con $|d - d_i| > 3$ días para todo incendio d_i na mesma estación e.

A **ventá de exclusión de $\pm$3 días** responde a que os días contiguos a un incendio teñen meteoroloxía case idéntica á do propio día do incendio; etiquetalos como "non incendio" confundiría ao modelo (mesmas condicións, etiqueta oposta). As datas dos propios incendios mantense como positivas, non se exclúen.

Variables derivadas

Calculamos 10 **variables derivadas** (en inglés *features*) de contexto temporal: días consecutivos sen choiva, precipitación acumulada (7/14/30 días), tendencias de temperatura e humidade (3 días), amplitude térmica, mes e día do ano. Estas variables calcúlanse **sobre a serie temporal completa de cada estación** (antes de filtrar a ventá de exclusión); en caso contrario, os cálculos sobre os últimos 7, 14 e 30 días terían ocos artificiais e darían valores incorrectos.

Resultado

O dataset final contén **93.221 filas** (4.512 positivos, 88.709 negativos, ratio 19,7:1) con **22 variables preditoras** (12 meteorolóxicas orixinais + 10 derivadas). O algoritmo completo en pseudocódigo recóllese no anexo A; o proceso cronolóxico de construción, coas decisións tomadas día a día e os problemas atopados, recóllese no **caderno de campo** (apéndice C).

4.5 Estratexia de modelado

O desbalanceo de clases (20:1) abordámolo con `class_weight='balanced'` e optimización do limiar de decisión (`TunedThresholdClassifierCV`), estratexia recomendada por **(author?)** [13] fronte a alternativas máis complexas.

Utilizamos unha **división temporal**: 2005–2018 como adestramento e 2019–2021 como test. Simula o escenario real (predicir o futuro con datos do pasado) e evita a **filtración temporal de información** (*data leakage temporal* en inglés, motivada no anexo A).

Avaliamos os modelos con PR-AUC, F1, Recall e Precision (definidas no apéndice B; xustificación da elección na sec. 5.2).

As ferramentas, bibliotecas e configuración de reproducibilidade (semente aleatoria `random_state=42`) detállanse no anexo A.

Capítulo 5

Resultados

5.1 Descrición do dataset

O dataset final (`dataset_modelo.csv`) ten as características que mostra a táboa 5.1.

Táboa 5.1: Características principais do dataset final.

Característica	Valor
Filas totais	93.221
Positivos (incendio)	4.512 (4,8%)
Negativos (non incendio)	88.709 (95,2%)
Variables preditoras	22
Rango temporal	2005–2021
Estacións meteorolóxicas	20

A división temporal produciu un conxunto de adestramento de 72.803 filas (2005–2018, con 4.157 incendios) e un conxunto de test de 20.418 filas (2019–2021, con 355 incendios). O test ten menos incendios proporcionalmente (ratio 56:1 fronte a 17:1 no conxunto de **adestramento**, en inglés *train*), o que o fai máis esixente.

5.2 Modelo de referencia (*baseline*): Logistic Regression

Cun desbalanceo de clase de 1:20 a **exactitude** (en inglés *accuracy*) non é unha métrica adecuada, porque premia acertar a clase maioritaria. PR-AUC, F1, recall e precision miden directamente o que importa: a calidade na detección dos incendios (clase minoritaria).

A Logistic Regression establece a referencia mínima (táboa 5.2).

Táboa 5.2: Métricas Logistic Regression (limiar optimizado, 0,786).

Métrica	Valor
F1-score	0,1593
Recall	0,3915
Precision	0,1000
PR-AUC	0,0883

O modelo detecta o 39% dos incendios, cunha precisión moi baixa: de cada 100 alertas, só 10 corresponden a incendios reais. O PR-AUC de 0,0883 é 5 veces superior ao azar (0,017), o que indica que o modelo si aprende patróns, pero de forma limitada.

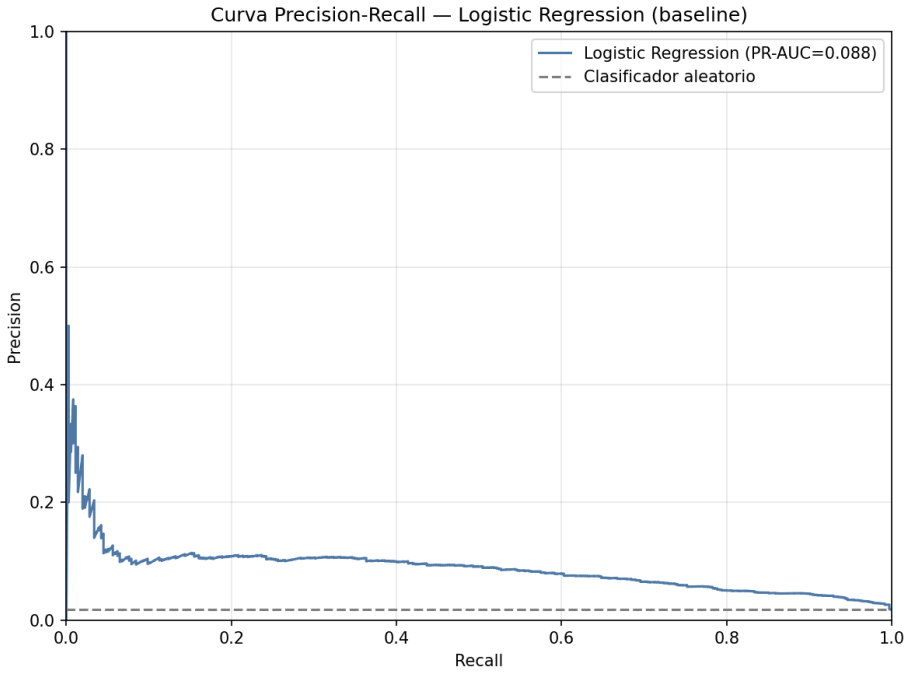


Figura 5.1: Curva precisión-recall do modelo Logistic Regression sobre o conxunto de test (2019–2021). A liña descontinua marca a precisión do clasificador aleatorio (1,7 %). PR-AUC = 0,0883.

5.3 Modelo principal: Random Forest

O Random Forest (300 árbores, profundidade máx. 15, mínimo 10 mostras/folla, pesos de clase balanceados; xustificación no anexo A) mellora substancialmente o modelo de referencia

(táboa 5.3).

Táboa 5.3: Métricas Random Forest (limiar optimizado, 0,613).

Métrica	**Valor**
F1-score	0,2082
Recall	0,4225
Precision	0,1381
PR-AUC	0,1485

5.4 Comparación de modelos

Táboa 5.4: Comparación de modelos: Logistic Regression vs. Random Forest.

Métrica	**Logistic Regression**	**Random Forest**	**Mellora**
PR-AUC	0,0883	0,1485	**+68,2%**
F1 (limiar opt.)	0,1593	0,2082	+30,7%
Recall	0,3915	0,4225	+7,9%
Precision	0,1000	0,1381	**+38,1%**

Random Forest mellora todas as métricas á vez: o salto principal está no **PR-AUC (+68 %)**, que mide a capacidade discriminativa independentemente do limiar, e na **precisión (+38 %)**, que reduce as falsas alarmas mantendo o *recall*.

5.5 Importancia de variables

As variables de **seca** (días sen choiva, precipitación acumulada) e **humidade** dominan o ranking. A temperatura, intuitivamente o factor máis obvio, queda en posicións intermedias, resultado coherente coa ciencia forestal: a auga dispoñible no chan e no aire é o que decide se o combustible vexetal pode arder.

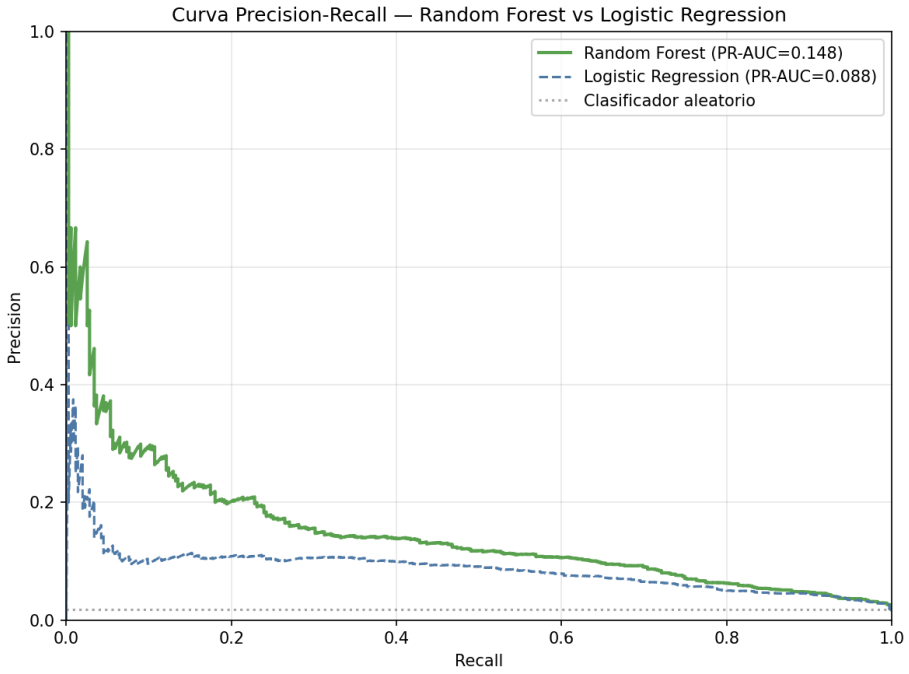


Figura 5.2: Comparación das curvas precisión-recall de Random Forest (verde) e Logistic Regression (azul) sobre o conxunto de test. Random Forest domina sistemáticamente a Logistic Regression en todo o rango de *recall*.

Táboa 5.5: Cinco variables máis importantes segundo Random Forest (ránking completo no anexo A).

Pos.	Variable	Importancia	Interpretación
1	Días sen choiva	0,120	Seca acumulada
2	Precip. acumulada 7 días	0,105	Choiva recente
3	Humidade mínima 3 días	0,095	Humidade sostida
4	Humidade relativa media	0,081	Humidade do aire
5	Precip. acumulada 14 días	0,074	Seca a medio prazo

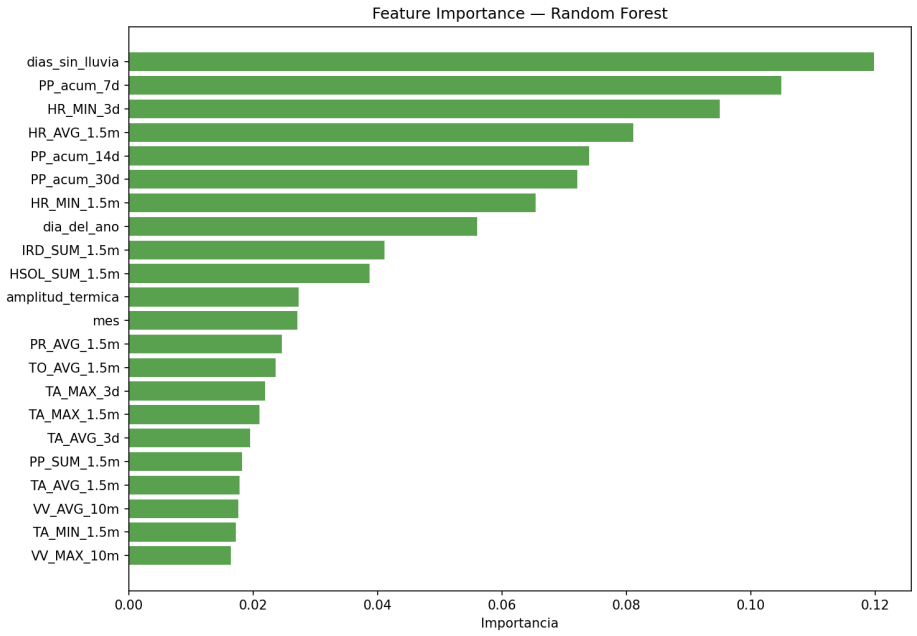


Figura 5.3: Importancia das 22 variables segundo o modelo Random Forest. As seis primeiras son indicadores de seca e humidade, coherente coa ciencia forestal e co sistema FWI canadense.

Capítulo 6

Discusión

6.1 Validación da hipótese

Os resultados confirman a hipótese de partida: as condicións meteorolóxicas conteñen información suficiente para distinguir, cun rendemento significativamente superior ao azar, entre días con e sen incendio. O modelo Random Forest discrimina 9 veces mellor que un clasificador aleatorio (PR-AUC = 0,15 fronte a 0,017).

O modelo non predí o evento "vaise producir un incendio", senón a probabilidade de que se den as condicións meteorolóxicas favorables para que un incendio se propague (de existir unha ignición). Esta distinción importa para interpretar os falsos positivos e falsos negativos: un día con risco predito alto sen incendio rexistrado non é necesariamente un erro do modelo, pode reflectir simplemente a ausencia dunha ignición —factor non meteorolóxico—; do mesmo xeito, un incendio en condicións desfavorables (intencionado, neglixencia eléctrica…) tampouco refuta a hipótese.

Os valores absolutos de PR-AUC (0,15) e F1 (0,21) son baixos, pero coherentes co estado da arte na predición de incendios con só meteoroloxía (F1 típicos de 0,15-0,30 na literatura). Non concebimos o modelo como clasificador binario, senón como score continuo de risco: ordena os días e deixa que o operador (Xunta, bombeiros) elixa o limiar segundo o custo relativo entre falsas alarmas e incendios non detectados. A discriminación 9× sobre o azar abonda para esa función. É a lóxica de LUMIA, que mostra para cada estación a probabilidade puntual e a súa categoría operativa (Baixo / Medio / Alto / Moi Alto), patrón equivalente ao do FWI canadense ou o IRDI da Xunta. Alcanzar F1 alto requiriría información de vexetación, topografía e actividade humana ausente do dataset.

6.2 Limitacións do enfoque

A meteoroloxía non é suficiente por si soa. Os incendios dependen tamén de vexetación, topografía e actividade humana, factores ausentes no dataset; en Galicia máis do 76 % son intencionados, polo que a compoñente humana é especialmente relevante.

Conxunto de test difícil e resolución espacial limitada. 2019–2021 ten ratio de desbalanceo 56:1 (fronte a 17:1 no adestramento), o que penaliza as métricas. Ademais, a estación máis próxima está a unha mediana de 10,1 km do incendio, e os microclimas de montaña poden diferir dos seus rexistros.

Sesgo do EGIF e degradación futura. O EGIF só recolle incendios atendidos polos servizos de extinción, polo que o modelo aprende a predicir os significativos. Ademais, adestrámolo con 2005–2018: nun escenario de cambio climático con eventos extremos crecentes (incendios de 2025), o modelo pode subestimar o risco real.

6.3 Coherencia coa literatura científica

A importancia das variables obtida polo Random Forest é coherente coa ciencia forestal: a seca e a humidade son os factores primarios, seguidos da estacionalidade. Isto coincide co deseño do propio sistema FWI canadense, que outorga un papel central ao contido de humidade do combustible.

A superioridade do Random Forest sobre a Logistic Regression (+68 % en PR-AUC) confirma ademais que as relacións son non lineais: non abonda con "máis calor = máis risco".

Capítulo 7

Conclusións

7.1 Conclusións do proxecto

1. Construímos un **dataset propio** de 93.221 observacións cruzando datos de incendios forestais (Civio/EGIF) con datos meteorolóxicos diarios de MeteoGalicia. Procesamos o 84,6% (6.528) dos 7.719 incendios rexistrados en Ourense entre 2005 e 2021 que tiñan **datos completos de temperatura, humidade e precipitación no día do incendio**; o 15,4% restante (1.191 incendios) descartámolos para evitar inventar valores meteorolóxicos nun día que o modelo usa para aprender.

2. O modelo Random Forest **supera significativamente ao modelo lineal** (Logistic Regression), o que confirma que as relacións entre meteoroloxía e incendios son non lineais e xustifica o uso de modelos baseados en árbores de decisión.

3. Os **indicadores de seca** (días consecutivos sen choiva, precipitación acumulada) e a **humidade relativa** son as variables máis relevantes para a predición de incendios, resultado coherente coa literatura científica forestal e co deseño do sistema FWI canadense, no que se basea o IRDI publicado pola Xunta de Galicia.

4. Os valores absolutos do modelo son modestos pero esperables (PR-AUC 0,15, F1 0,21): o tope vén das variables non meteorolóxicas ausentes (vexetación, topografía, factor humano), non do método empregado.

7.2 Reflexión sobre o proceso de investigación

Este proxecto foi unha aprendizaxe integral en ciencia de datos: Pasamos por todo: obter e limpar datos reais (coas súas inconsistencias), tomar decisións baseadas na literatura científica e avaliar modelos. A construción do dataset propio, coa resolución de problemas como a conversión de coordenadas, a estratexia de respaldo para estacións sen datos e o manexo do

desbalanceo de clases, foi a parte máis formativa do proxecto. O percorrido completo paso a paso, coas decisións e os erros atopados polo camiño, está documentado no **caderno de campo** (apéndice C).

7.3 Difusión: a interface LUMIA

Como complemento ao modelado, publicamos a aplicación web **LUMIA** (*Lume + IA*, `https://lumia.cabanelas.info/`): predición do risco diario por estación, exploración do histórico nun mapa interactivo e predictor manual.

7.4 Proposta de continuidade

- **Variables non meteorolóxicas**: NDVI (índice de vexetación por satélite), topografía e uso do solo, e indicadores indirectos da actividade humana (calendario agrícola, festivos), maior fonte de variación non capturada (76 % dos incendios en Galicia son intencionados).
- **Modelos máis avanzados**: XGBoost ou LightGBM, que na literatura superan a Random Forest en datos tabulares.
- **Validación operativa**: probar LUMIA con bombeiros forestais e distritos forestais.
- **Ampliación xeográfica**: estender o *pipeline* (a cadea completa de procesado) ás 155 estacións de MeteoGalicia en toda Galicia.

Apéndices

Apéndice A

Detalles metodolóxicos

A.1 Conversión de coordenadas UTM–WGS84

Os datos de incendios e as estacións meteorolóxicas veñen en sistemas de coordenadas distintos, polo que para calcular distancias en metros entre eles é preciso pasar todo a un sistema común. Os datos de incendios da Fundación Civio veñen en **latitude/lonxitude (WGS84)**, o sistema estándar de coordenadas xeográficas usado polos dispositivos GPS. As estacións de MeteoGalicia están en **UTM zona 29N**, o sistema cartográfico oficial para Galicia, no que as coordenadas se miden en metros. Implementamos a conversión exacta seguindo as fórmulas oficiais de **(author?)** [10], sen depender de bibliotecas externas como `pyproj`, para entender en detalle o que se estaba a calcular.

A.2 Estratexia de división temporal vs. aleatoria

A división temporal (2005–2018 adestramento, 2019–2021 test) é a única xeneralizable a un sistema de produción real, no que mañá só se coñecen os datos de hoxe e do pasado. Unha división aleatoria, en cambio, mesturaría días de 2005 con días de 2021, e o modelo podería aprender patróns do futuro presentes nas variables de tendencia (precipitación acumulada en 7–30 días, días consecutivos sen choiva), invalidando a avaliación. Este fenómeno, coñecido como **filtración temporal de información** (*data leakage temporal* en inglés), é unha das fontes máis frecuentes de avaliación inflada en proxectos de *machine learning* con series temporais.

A.3 Configuración do Random Forest

Escollemos a configuración final do Random Forest (300 árbores, `n_estimators=300`; profundidade máxima 15, `max_depth=15`; mínimo 10 mostras por folla, `min_samples_leaf=10`;

pesos de clase balanceados, `class_weight='balanced'`) buscando un equilibrio entre aprender ben os patróns dos datos de adestramento e evitar o **sobre-axuste** (que o modelo memorice casos concretos en vez de aprender patróns xerais):

- **300 árbores**: con esa cantidade a votación do conxunto xa é estable; máis árbores apenas mellorarían o resultado.
- **Profundidade máxima de 15 niveis**: limita o número de preguntas encadeadas que cada árbore pode facer, evitando que memoricen casos concretos en vez de aprender patróns xerais.
- **Mínimo de 10 mostras por folla**: cada decisión final do árbore está sustentada por polo menos 10 días reais, polo que se evitan regras baseadas só nun ou dous casos.
- **Pesos de clase balanceados**: ponderan os erros sobre incendios case 20 veces máis que sobre días sen incendio, para que o modelo non os ignore por seren raros.

A.4 Métricas de avaliación e PR-AUC

Non utilizamos a exactitude porque en datos desbalanceados un clasificador trivial que sempre di "non incendio" obtería un 95 % de exactitude sen detectar ningún incendio. As métricas elixidas (PR-AUC, F1-score, Recall e Precision; definidas no apéndice B) reflicten o equilibrio real entre detección e falsas alarmas. Seguimos as recomendacións de **(author?)** [13]: combinar `class_weight='balanced'` con optimización do limiar de decisión (a clase `TunedThresholdClassifierCV` de *scikit-learn*, que busca o limiar óptimo mediante **validación cruzada**: divide os datos de adestramento en varios bloques, entrena varias veces variando que bloque queda fóra para avaliar, e elixe o limiar que mellor funciona en media), estratexia que en 9.000 experimentos sobre 30 datasets superou a técnicas alternativas máis complexas.

A.5 Importancia completa das 22 variables

A táboa A.1 reúne a importancia por impureza (definida no apéndice B) de tódalas variables segundo Random Forest.

Táboa A.1: Importancia das 22 variables segundo Random Forest.

Pos.	Variable	Importancia
1	Días sen choiva	0,120
2	Precipitación acumulada 7 días	0,105
3	Humidade mínima 3 días	0,095
4	Humidade relativa media	0,081
5	Precipitación acumulada 14 días	0,074
6	Precipitación acumulada 30 días	0,072
7	Humidade relativa mínima	0,065
8	Día do ano	0,056
9–22	Resto de variables (Temp., presión, vento, sol, etc.)	< 0,05 cada

A.6 Algoritmo de construción do dataset (pseudocódigo)

O algoritmo implementado en `src/construir_dataset_clasificacion.py` resúmese na seguinte secuencia de pasos. As constantes son: `EXCLUSION_WINDOW_DAYS` = 3 (vétanse os días ± 3 arredor de cada incendio para non confundir o modelo: teñen meteoroloxía case idéntica ao día do incendio pero etiqueta oposta) e `DRY_THRESHOLD` = `1,0 mm/día` (limiar para considerar un día seco).

```
ENTRADA:
    meteo         -- DataFrame con (fecha, estacion_id, 12 vars
    ↪ meteorolóxicas)
    fusionado     -- DataFrame con incendios xa asignados a estación
    estaciones    -- Catálogo das 20 estacións

SAÍDA:
    dataset_modelo  -- DataFrame con 93.221 filas e 22 features +
    ↪ etiqueta y

PASO 1 — Identificar pares (data, estación) con incendio
    fire_pairs = { (data_i, estacion_i) para cada incendio i en
    ↪ fusionado }
```

```
PASO 2 — Construír ventá de exclusión ±3 días
    exclusion_pairs = { }
    PARA cada (data, estacion) en fire_pairs:
        PARA delta en {-3, -2, -1, 0, +1, +2, +3}:
            exclusion_pairs.add((data + delta, estacion))
    exclusion_only = exclusion_pairs - fire_pairs   // os incendios
    ↪ mantéñense como +

PASO 3 — Etiquetar cada fila meteorolóxica
    PARA cada fila (data, estacion) en meteo:
        SE (data, estacion) en fire_pairs:    etiqueta = +1   //
    ↪ positivo
        SENÓN SE (data, estacion) en exclusion_only:
                                              etiqueta = -1   //
    ↪ descartar
        SENÓN:                                etiqueta =  0   //
    ↪ negativo

PASO 4 — Calcular 10 features derivadas (POR ESTACIÓN, sobre serie
    ↪ completa)
    PARA cada estacion e en estaciones:
        serie = meteo[estacion = e] ordenada por data
        serie["dias_sen_choiva"]   = días consecutivos con PP < 1,0 mm
        serie["PP_acum_7d"]        = serie["PP"].rolling(7).sum()
        serie["PP_acum_14d"]       = serie["PP"].rolling(14).sum()
        serie["PP_acum_30d"]       = serie["PP"].rolling(30).sum()
        serie["TA_AVG_3d"]         = serie["TA_AVG"].rolling(3).mean()
        serie["TA_MAX_3d"]         = serie["TA_MAX"].rolling(3).max()
        serie["HR_MIN_3d"]         = serie["HR_MIN"].rolling(3).min()
        serie["amplitud_termica"]  = serie["TA_MAX"] - serie["TA_MIN"]
        serie["mes"]               = serie["data"].mes
        serie["dia_do_ano"]        = serie["data"].día_do_ano

    // IMPORTANTE: o rolling faise ANTES de filtrar a exclusión
    // para que non teña ocos artificiais.

PASO 5 — Filtrar filas excluídas e gardar
    dataset_modelo = meteo[etiqueta != -1]
    columnas_finais = [data, estacion_id, etiqueta, 12_meteo + 10
    ↪ _derivadas]
    gardar(dataset_modelo, "dataset_modelo.csv")
```

Algoritmo de predición

A predición sobre un caso novo é sinxela: o *pipeline* de *scikit-learn* (unha cadea de pasos encadeados, gardada en `rf_pipeline.pkl`) encadea dúas etapas. Primeiro `SimpleImputer` substitúe os valores faltantes (por exemplo, días sen lectura de humidade) pola mediana correspondente do conxunto de adestramento —o que se chama **imputación**. Despois `RandomForestClassifier` aplica o bosque e devolve unha probabilidade en $[0, 1]$.

```
ENTRADA:
    features  -- vector de 22 valores meteorolóxicos (poden conter None)

SAÍDA:
    prob      -- probabilidade en [0, 1] de que se den condicións
                 favorables a incendio

PASO 1 — Imputación: substituír valores None por mediana do adestramento
    features = imputer.transform(features)

PASO 2 — Inferencia con Random Forest (300 árbores)
    PARA cada árbore t en {1, ..., 300}:
        voto_t = árbore_t.predict(features)   // voto en {0, 1}
    prob = media(voto_1, ..., voto_300)        // proporción de votos a
    ↪ 1

PASO 3 — Clasificación binaria (opcional, segundo limiar)
    SE prob >= limiar (por defecto 0,613):
        decisión = "condicións favorables a incendio"
    SENÓN:
        decisión = "sen condicións favorables"

DEVOLVER (prob, decisión)
```

O limiar 0,613 escolleuse mediante `TunedThresholdClassifierCV` sobre o conxunto de adestramento, optimizando o F1-score (anexo A, sección de métricas).

A.7 Ferramentas e reproducibilidade

- **Python 3.10+** como linguaxe de programación.
- **pandas** e **NumPy** para manipulación e análise de datos.
- **scikit-learn** ($\geq$1.5) [8], incluíndo `TunedThresholdClassifierCV`.
- **matplotlib** para a xeración de gráficos.
- **requests** para a descarga de datos da API de MeteoGalicia.

- **LaTeX** para a redacción da memoria.

Para garantir a reproducibilidade, fixamos a semente aleatoria (`random_state=42`) en todos os procesos do modelo nos que interveñen elementos aleatorios (creación das árbores do Random Forest, optimización do limiar de decisión e división dos datos).

Conxunto de datos público

O dataset final (`dataset_modelo.csv`, 93.221 filas $\times$ 22 variables preditoras + etiqueta) está dispoñible publicamente para descarga e verificación:

drive.google.com/file/d/1LLWisILDPcHog7M2TWFMB14q5dqckRVm

Apéndice B

Regresión Loxística e Bosques Aleatorios

No ámbito da **aprendizaxe automática supervisada** (adestramos o modelo con exemplos nos que xa coñecemos a resposta), o obxectivo é construír un modelo capaz de predicir o valor dunha variable resposta y a partir dun conxunto de variables preditoras $x_1, x_2, \ldots, x_p$, utilizando para iso un conxunto de datos de adestramento con n observacións. Neste apéndice describimos os dous modelos empregados no presente traballo: a **regresión loxística** e os **bosques aleatorios** (*Random Forest*). Ambos resolven o mesmo tipo de problema —decidir se un día terá ou non un incendio—, pero difiren substancialmente: parten de **supostos diferentes** sobre como se relacionan as variables, **representan relacións de tipo distinto** (lineais nun caso, non lineais e con interaccións no outro) e ofrecen **niveis distintos de transparencia** sobre por que dan unha predición concreta.

B.1 Regresión Loxística

A **regresión loxística** é un modelo de clasificación binaria: dado un conxunto de valores das variables preditoras, estima a probabilidade de que a observación pertenza á clase positiva (no noso caso, "día con incendio"). Para iso transforma unha combinación lineal das variables preditoras a través da función *loxística* (ou *sigmoide*), que comprime calquera número real ao intervalo $(0, 1)$, que é o rango natural dunha probabilidade. Esta hipótese supón que a relación entre os preditores e as *log-odds* (o logaritmo do cociente entre a probabilidade da clase positiva e a da negativa) da resposta é lineal, o que simplifica a estimación e permite unha interpretación clara dos coeficientes.

Supoñamos que dispomos de n observacións con p preditores e etiquetas $y \in \{0, 1\}$.

Definamos a combinación lineal das variables preditoras como

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p,$$

onde β_0 é o termo independente e $\beta_1, \beta_2, \ldots, \beta_p$ son os coeficientes asociados a cada predictor. O modelo exprésase como

$$P(y = 1) = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Para ver con máis claridade a estrutura lineal, podemos despexar z. Chamando $p = P(y = 1)$:

$$p = \frac{1}{1 + e^{-z}} \iff e^{-z} = \frac{1 - p}{p} \iff z = \log\left(\frac{p}{1 - p}\right).$$

A cantidade $\log(p/(1 - p))$ é a chamada *log-odds*, e o que asume a regresión loxística é que esta cantidade é unha combinación lineal das variables preditoras.

Estimación dos parámetros. Estimamos os coeficientes buscando aqueles que fan que as predicións do modelo se axusten o mellor posible aos datos reais do adestramento. Matematicamente, isto equivale a minimizar unha función de perda chamada **entropía cruzada** (tamén coñecida como *log-loss* en inglés). A idea intuitiva: a entropía cruzada penaliza máis canto máis lonxe estea a probabilidade predita da etiqueta real. Para unha observación positiva ($y = 1$), se o modelo asigna probabilidade próxima a 1 a perda é pequena; se a asigna próxima a 0, a perda dispárase. Súmase para todas as observacións do adestramento. O problema ten un único mínimo posible, polo que existe unha solución óptima. Para atopala empregamos un método iterativo chamado *descenso do gradiente*, que axusta paso a paso os coeficientes na dirección que máis reduce a perda.

Avaliación do modelo. En clasificación binaria con clases desbalanceadas (no noso conxunto de datos a clase positiva supón apenas o $4{,}8\,\%$ das observacións, ratio $19{,}7{:}1$), a exactitude resulta enganosa: un clasificador trivial que sempre prediga "non incendio" obtería un $\sim$95 % de exactitude sen detectar ningún incendio. Empregamos polo tanto métricas máis informativas:

- Precision e recall. Definamos como **verdadeiros positivos** os incendios correctamente predichos polo modelo, **falsos positivos** as alertas que non corresponden a un incendio real, e **falsos negativos** os incendios reais que o modelo non detectou. Entón *precision* = verdadeiros positivos / (verdadeiros positivos + falsos positivos) é a fracción de alertas correctas; *recall* = verdadeiros positivos / (verdadeiros positivos + falsos negativos) é a fracción de incendios efectivamente detectados.

- F1-score. Combina precision e recall nun único número mediante a súa **media harmónica**: $F_1 = 2\,\text{Precision} \cdot \text{Recall}/(\text{Precision} + \text{Recall})$. A diferenza da media aritmética, a media harmónica penaliza fortemente os desequilibrios. Por exemplo, se precision = 0,9 e recall = 0,1, a media aritmética dá 0,5 (parece razoable), pero a media harmónica (F1) dá apenas 0,18. Isto reflicte que un modelo que detecta moi poucos incendios reais non é útil, por moi seguras que sexan as poucas alertas que emite.

- PR-AUC. O modelo non dá unha clasificación binaria directa, senón unha probabilidade (entre 0 e 1) de que sexa un día con incendio. Para convertela en si/non hai que escoller un **limiar de decisión**: por exemplo, "marcar como incendio se a probabilidade é superior a 0,5". Cada limiar dá un par diferente (precision, recall): un limiar baixo detecta moitos incendios reais pero produce moitas alertas falsas; un limiar alto produce poucas alertas falsas pero deixa pasar incendios. A **curva Precision-Recall** debuxa estes pares para todos os limiares posibles, e a **PR-AUC** é a área que queda baixo a curva: un único número entre 0 e 1 que resume o rendemento do modelo ao longo de todos os limiares. É a métrica principal en datos desbalanceados, xa que se centra no rendemento sobre a clase minoritaria.

- Entropía cruzada. A propia función de perda da regresión loxística. A diferenza das anteriores, non só ten en conta se a clasificación é correcta, senón tamén canta seguridade ten o modelo na súa predición: penaliza máis cando o modelo predí cunha probabilidade alta de incendio (por exemplo, 95 %) e despois resulta que non hai incendio.

Regularización. Cando o número de preditores é elevado ou hai variables preditoras moi correlacionadas entre si, o modelo pode aprender demasiado os detalles dos datos de adestramento (incluso o ruído) e fallar en datos novos: este fenómeno coñécese como **sobre-axuste** (*overfitting* en inglés). Para evitalo penalízase a magnitude dos coeficientes do modelo, é dicir, oblígase a que se manteñan pequenos durante o axuste. Neste proxecto empregamos a chamada *regularización por suma de cadrados* dos coeficientes (coñecida na bibliografía anglosaxona como *Ridge*), que é a opción por defecto de *scikit-learn*. A idea é sinxela: á función de perda (a entropía cruzada) súmaselle un termo que medra canto máis grandes sexan os coeficientes do modelo, polo que o axuste premia coeficientes pequenos. Iso impide que un único predictor domine a predición só por ter unha magnitude grande, e estabiliza o modelo cando hai preditores correlacionados entre si. A intensidade da penalización contrólase cun único parámetro que se elixe antes do adestramento.

Existen outras variantes de regularización —en inglés *Lasso* e *Elastic Net*— que neste proxecto non se utilizaron. A súa principal diferenza é que penalizan os coeficientes doutro xeito, o que ten como efecto adicional poder anular algúns por completo e realizar así unha selección automática de variables.

A regresión loxística presenta como principais vantaxes a súa alta **interpretabilidade** (é dicir, a facilidade para entender por que o modelo dá unha predición concreta) —os coeficientes $\hat{\beta}_j$ cuantifican o efecto de cada predictor sobre as *log-odds* da resposta— e a súa eficiencia computacional, mesmo en datasets de tamaño grande. Como inconveniente fundamental, supón que a relación entre os preditores e as *log-odds* é lineal, polo que non captura interaccións nin non linealidades sen unha transformación previa das variables (por exemplo, crear novas variables a partir das orixinais que xa codifiquen non linealidades ou combinacións entre preditores), o que pode limitar a súa capacidade preditiva en problemas complexos.

B.2 Bosques Aleatorios (*Random Forest*)

Os **bosques aleatorios** (*Random Forest*) son un método de **aprendizaxe por conxunto** proposto por Breiman [**(year?)**]. A idea central é combinar un gran número de **árbores de decisión** entrenadas de forma independente sobre distintas submostras dos datos, promediando as súas predicións para obter un resultado final máis robusto e menos sensible a pequenos cambios nos datos de adestramento que o producido por un único árbore. Neste proxecto empregámolos para clasificación binaria.

Un **árbore de decisión** divide os datos facendo unha cadea de preguntas binarias sobre as variables preditoras: "é a humidade menor que 40 %?", "leva máis de 15 días sen chover?", e así sucesivamente. En cada paso, o algoritmo escolle automaticamente a pregunta —é dicir, o predictor x_j e o limiar t— que mellor separa as observacións segundo a súa clase. A *impureza* dun nodo mide canto están mesturadas a clase positiva e a negativa nesa rama: un nodo é puro cando todas as súas observacións pertencen á mesma clase. Para clasificación binaria a medida máis empregada é a **entropía**:

$$H = -p \log_2(p) - (1-p) \log_2(1-p),$$

onde p é a proporción de observacións da clase positiva ("incendio") no nodo.

A entropía toma o seu valor **máximo** ($H = 1$ bit) cando $p = 0{,}5$, é dicir, cando o nodo está completamente mesturado e ten 50 % de cada clase. Toma o seu valor **mínimo** ($H = 0$) cando $p = 0$ ou $p = 1$, é dicir, cando o nodo é puro e todas as súas observacións pertencen á mesma clase. A intuición vén da **teoría da información**: a entropía mide a incerteza sobre a clase dunha observación elixida ao azar do nodo. Un nodo puro non ten incerteza ningunha; un nodo mesturado a partes iguais ten a máxima incerteza posible.

O algoritmo escolle, en cada nodo, a división que reduce máis a entropía total dos seus fillos respecto ao pai. Este descenso de entropía chámase *ganancia de información* e expresa canta certeza gañamos sobre a clase ao realizar a división. Cando ningunha división posible reduce significativamente a entropía, o algoritmo deixa de partir o nodo.

Nota sobre a implementación. O *scikit-learn* emprega por defecto outra medida de impureza chamada *índice de Gini*, que se comporta de xeito moi parecido á entropía: tamén é mínima nun nodo puro e máxima nun nodo a partes iguais entre as dúas clases. Na práctica as dúas medidas dan árbores moi similares e métricas finais case idénticas, polo que neste traballo non se considerou necesario cambiar a opción por defecto.

Algoritmo de construción. Denotemos por B o número de árbores e por m o número de preditores candidatos en cada división. O procedemento é o seguinte.

1. Para cada $b = 1, \ldots, B$:
   (a) Obtense unha mostra $\mathcal{D}_b^*$ de tamaño n extraída con reposición a partir de $\mathcal{D}$ (algunhas observacións aparecerán repetidas e outras quedarán fóra dese subconxunto).
   (b) Adéstrase un árbore de decisión T_b sobre $\mathcal{D}_b^*$. En cada nodo, selecciónanse aleatoriamente $m \leq p$ preditores candidatos e elíxese a mellor división *só entre eles*.
   (c) Faise crecer o árbore ata a profundidade máxima, sen recortar ramas despois.
2. Combínanse as B predicións. Para clasificación (o noso caso) aplícase a **votación maioritaria**: cada árbore "vota" pola clase que predí para a observación, e a clase final é a máis votada. En problemas de regresión calcularíase a media aritmética das predicións dos B árbores en lugar da clase máis votada.

A dobre fonte de aleatoriedade —mostraxe con reposición das observacións e selección aleatoria dos preditores— reduce a correlación entre árbores. Isto é clave: cando os árbores son moi parecidos entre si (alta correlación), promedialos non mellora moito o resultado individual; cando son distintos (baixa correlación), o promedio cancela os erros aleatorios de cada árbore e o conxunto faise máis robusto. Por iso minimizar a correlación entre árbores mediante un m pequeno é a principal panca para reducir o erro.

Erro sobre as mostras non seleccionadas (OOB). Dado que cada árbore se adestra sobre unha mostra extraída con reposición, en media aproximadamente o $36{,}8\,\%$ das observacións do conxunto orixinal non son seleccionadas para o adestramento dese árbore. Estas observacións permiten estimar o erro de xeneralización sen necesidade dun **conxunto de validación** separado (un subconxunto de datos que normalmente se aparta antes do adestramento para avaliar o rendemento do modelo despois): para cada observación empregamos só os árbores nos que non foi seleccionada, promediamos as súas predicións, comparamos coa etiqueta real e contabilizamos os erros. A taxa de erro resultante é a estimación OOB (*out-of-bag* en inglés, nome co que aparece en bibliografía e en *scikit-learn*).

Importancia das variables. Os bosques aleatorios proporcionan dúas medidas de relevancia dos preditores. Neste proxecto empregamos a primeira:

Importancia por impureza (en inglés *Mean Decrease Impurity*). É a medida que devolve por defecto *scikit-learn* (atributo `feature_importances_`). Funciona así: cada vez que un predictor x_j se utiliza para dividir un nodo nun árbore, a división produce unha redución da impureza do nodo (medida pola entropía). Esa redución pondérase polo número de observacións que pasan polo nodo, e súmase para todos os nodos onde x_j aparece, en todos os árbores do bosque. Os preditores cuxas divisións reducen máis impureza son os máis relevantes para o modelo. É unha medida rápida de calcular —xa se computa durante o adestramento—, pero pode presentar un sesgo favorable aos preditores con moitos valores distintos (por exemplo, variables continuas como temperatura ou precipitación, fronte a variables categóricas con poucos valores). No noso caso, a maioría das 22 variables son continuas, polo que este sesgo pode afectar lixeiramente o ranking; aínda así, o resultado xeral (seca e humidade como factores dominantes) é coherente coa ciencia forestal.

Alternativamente, a **importancia por permutación** (en inglés *Mean Decrease Accuracy*) permuta aleatoriamente os valores dun predictor sobre as mostras non seleccionadas (OOB) e mide canto cae o rendemento do modelo: un predictor relevante provoca unha caída notable, un irrelevante apenas modifica o resultado. É máis robusta a este sesgo pero moito máis custosa de calcular, e neste proxecto non se utilizou.

Os bosques aleatorios teñen varias vantaxes prácticas. A primeira é a súa alta precisión preditiva sen requirir un preparado coidado dos datos: a diferenza dos modelos lineais, non precisan que as variables estean nunha escala común (temperaturas en graos e precipitacións en mm conviven sen problema) nin transformacións previas. A segunda é a súa robustez fronte a valores extremos: como cada división se decide comparando valores cun limiar, un dato puntual moi grande ou moi pequeno non desestabiliza o modelo, simplemente vai parar a un nodo concreto. Tamén toleran ben preditores irrelevantes: as variables que non aportan información apenas se escollen para dividir nodos e quedan automaticamente descartadas. A estimación OOB integrada evita a necesidade de reservar un conxunto de validación, e o modelo é fácil de adestrar en paralelo: como cada árbore se constrúe de forma independente do resto, o adestramento pode distribuírse entre varios núcleos da CPU para reducir o tempo total. No noso caso, *scikit-learn* utiliza por defecto todos os núcleos dispoñibles. Como inconvenientes principais cómpre sinalar a menor interpretabilidade en comparación coa regresión loxística, o elevado custo en memoria e tempo de predición para valores grandes de B, e a tendencia a perder precisión cando se lle pide unha predición para valores das variables fóra do rango visto no adestramento (por exemplo, unha temperatura máis alta que calquera vista no histórico).

B.3 Comparación entre os dous modelos

A elección entre a regresión loxística e os bosques aleatorios depende do equilibrio desexado entre interpretabilidade e capacidade preditiva, así como dos supostos que sexa razoable manter sobre a estrutura dos datos. A regresión loxística asume unha forma matemática fixa: supón que existe unha relación lineal entre os preditores e as *log-odds* da resposta, e iso permite unha interpretación directa dos coeficientes en termos do efecto sobre a probabilidade da clase positiva. Os bosques aleatorios, pola contra, non parten de ningún suposto sobre a forma da relación: aprenden directamente dos datos as relacións non lineais e as interaccións entre preditores, a costa dunha menor transparencia no proceso de decisión.

Cada un dos dous modelos é máis adecuado en contextos distintos. A **regresión loxística** é preferible cando se quere unha interpretación clara de como cada variable afecta á predición, e cando é razoable supoñer que a relación entre preditores e probabilidade é aproximadamente lineal. Os **bosques aleatorios** ofrecen mellor rendemento cando a relación entre as variables é complexa ou implica interaccións entre preditores que un modelo lineal non pode capturar. Neste traballo empregamos ambos os modelos de forma complementaria: a regresión loxística como liña base e os bosques aleatorios como modelo principal, para contrastar un enfoque lineal cun non lineal sobre o mesmo problema.

Apéndice C

Caderno de campo

Proxecto: Predición de incendios forestais en Ourense mediante aprendizaxe automática **Autores**: Mikel Brea Gómez · Lúa Piñeiro Cabanelas · Dimitrii Rapoport Shakin **Titor do centro**: Carlos Ferreiro García (IES As Lagoas) **Cotitor universitario**: José Ángel Cid Arújo (Departamento de Matemáticas, Universidade de Vigo) **Programa**: STEMbach (Bacharelato de Excelencia en Ciencias e Tecnoloxía) **Bienio**: 2024 / 2026 **Centro**: IES As Lagoas, Ourense

C.1 Prólogo

Este caderno recolle, de forma cronolóxica, o desenvolvemento real do proxecto. Non é unha memoria final nin un manual técnico: é o diario do que fixemos, das decisións que tomamos, dos camiños que probamos e descartamos, dos problemas que apareceron e de como os resolvemos.

A diferenza da memoria oficial —onde só queda o resultado final—, aquí queda **o proceso**: as preguntas que nos fixemos, o que non sabiamos ao empezar, as veces que cambiamos de rumbo. Reflicte o método científico aplicado a un problema real con datos reais, e tamén as cousas que aprendemos polo camiño.

C.2 Sobre o uso de intelixencia artificial neste proxecto

Este proxecto é, en parte, sobre intelixencia artificial: queremos predecir incendios cun modelo de aprendizaxe automática. Pero a IA tamén foi unha **ferramenta de traballo** ao longo do desenvolvemento, e queremos ser transparentes sobre como.

Usamos **Claude (Anthropic)** como **executor técnico** das nosas decisións. Cando precisabamos un script en Python, unha consulta a unha API ou unha fórmula matemática implementada, describíamoslle á IA o que queriamos lograr e ela escribía o código. **O método de**

traballo foi sempre o mesmo: preguntamos, investigamos, avaliamos e decidimos. Despois pedímoslle á IA que executase a decisión técnica.

As **decisións sempre foron nosas**:

- Que problema atacar (incendios en Ourense)
- Que fontes de datos usar (Civio + MeteoGalicia)
- Como tratar os datos (filtros, rangos, interpolación)
- Que algoritmos probar (Logistic Regression, Random Forest)
- Como avaliar (métricas, división temporal, ventá de exclusión)
- Que conclusións tirar (limitacións, valor científico)

A IA nunca tomou decisións por nós, nin propuxo enfoques sen pedirllos. Foi unha ferramenta de produtividade, non un colaborador con criterio propio. Esta é a forma en que se está a usar hoxe a IA na investigación e na enxeñaría reais: **o cerebro humano dirixe; a IA executa**.

C.3 Paso 0 · Como nace o proxecto

2026 · IES As Lagoas, Ourense

O proxecto arrincou cunha proposta de **Carlos Ferreiro García**, profesor de Matemáticas e coordinador do programa STEMbach do centro: «entrenade un modelo de intelixencia artificial». A escolla do tema quedou nas nosas mans.

Discutimos varias liñas, todas relacionadas con **recoñecemento de imaxes**, ata que rotamos completamente cara a outra dirección: queriamos **algo útil para Galicia**. Os incendios forestais apareceron como o problema máis evidente, dado que Ourense é a provincia máis afectada de España.

Ao buscar conxuntos de datos atopamos que **as bases dispoñibles eran estranxeiras**: a máis coñecida, *UCI Algerian Forest Fires Dataset*, xa estaba modelada e con resultados publicados. Replicar un traballo sobre datos de Arxelia non tiña valor para nós.

Tomamos entón a decisión que marcou todo o resto do proxecto: **construír o noso propio dataset**. Pensamos primeiro en cubrir toda Galicia, pero o volume era inviable para o tempo dispoñible, así que reducimos o ámbito á provincia de **Ourense**. Cruzaríamos dúas fontes públicas: o rexistro histórico de incendios da Fundación Civio (orixe EGIF do Ministerio) e os datos meteorolóxicos diarios da API de MeteoGalicia.

Non establecemos roles fixos ao principio. Cada un foi asumindo as tarefas que mellor se axustaban ás súas fortalezas.

C.4 Paso 1 · As primeiras descargas

25 de marzo de 2026

O proxecto necesitaba dúas fontes de datos. Cada unha plantexou un reto distinto.

Datos de incendios — Civio. A descarga foi manual: o portal de datos abertos da Fundación Civio require encher un formulario cun nome e un correo persoal. Rexistramonos co enderezo (`lua@cabanelas.info`), aceptamos as condicións de uso e descargou o ficheiro `fires-all.csv`. Pesa **31 MB** e contén **292.181 incendios** rexistrados en toda España entre 1968 e 2023, orixe da Estatística Xeral de Incendios Forestais (EGIF) do Ministerio. Foi o noso primeiro contacto co tamaño real do problema: medio millón de eventos.

Datos meteorolóxicos — MeteoGalicia. Aquí o reto era distinto. MeteoGalicia ten unha API REST pública (sen API key) que devolve os datos diarios estación por estación. Pero descargar **20 estacións × 16 anos × 12 meses ≈ 3.800 peticións** a man era inviable. Decidimos programar un script en Python (`src/descargar_meteo.py`, 107 liñas) que recorre todas as estacións de Ourense, fai as chamadas á API cunha pausa de cortesía entre cada unha e garda os JSON resultantes en disco. A descarga total tardou unhas tres horas; o script imprime o seu propio progreso para poder vixiar que non falle.

Por que 2005–2021. Ao cruzar as dúas fontes detectamos un problema temporal: as estacións meteorolóxicas máis antigas de MeteoGalicia (serie 10xxx) empezaron a operar de forma estable hacia 2005, e o CSV de Civio chega ata 2021. Antes de 2005 non había cobertura meteorolóxica suficiente; despois de 2021 non había datos de incendios. **Quedaron 17 anos** de solapamento, o noso rango efectivo de traballo.

Decidimos descartar tamén as estacións meteorolóxicas máis novas (serie 19xxx): aínda que Ourense ten 14 delas, só cobren 6-7 anos, fronte aos 16-17 das 10xxx. Incluílas non aportaba dato relevante para un dataset de aprendizaxe automática.

C.5 Paso 2 · Comprobando o que tiñamos

finais de marzo de 2026

Antes de programar nada, dedicamos uns días a **mirar os datos a man**. Abrimos os JSON de MeteoGalicia e o CSV de Civio nunha folla de cálculo, aplicamos filtros e fomos contando. Esta fase, aparentemente lenta, foinos clave: deixounos en evidencia tres cousas que despois marcaron todo o pipeline.

1. Catro estacións non miden vento. Filtrando por variable e estación, descubrimos que Entrimo, Monte Medo, As Petarelas e a estación de Ourense (4 das 20) **non teñen anemómetro**. A decisión de que facer foi **unánime**: en vez de descartar esas 4 estacións (perdiamos cobertura espacial), prestariámoslles o vento da estación con anemómetro máis próxima

cunha distancia razoable. Así as 20 estacións siguen sendo comparábeis, todas as filas do dataset teñen as mesmas 12 variables, e o modelo non ten que aprender un esquema heteroxéneo. A homogeneidade pesou máis que a integridade local do dato de vento.

2. Os datos veñen con código de validación. MeteoGalicia entrega cada medida cun campo `lnCodigoValidacion` que indica a súa fiabilidade: 1 = dato válido orixinal, 5 = dato válido interpolado pola propia MeteoGalicia, e os códigos 0, 2, 3 e 9 son respectivamente dato sen validar, sospeitoso, erróneo ou non rexistrado. Decidimos desde o principio aceptar só os códigos **1 e 5** e descartar o resto. Esta regra quedou centralizada nunha constante (`VALID_CODES = {1, 5}` en `src/config.py`) para non ter que repetila en ningures. O resultado foi excelente: **o 99,5%** das medidas pasaron o filtro.

3. A causa do incendio non é fiable. O 76% dos rexistros de Civio ten causa marcada como «suposta». Pero como o noso modelo predí *condicións de risco*, non *causas*, esta limitación non nos afecta directamente. Tomamos nota igualmente, para a discusión final.

C.6 Paso 3 · Capando o ruído

primeira semana de abril de 2026

Os datos meteorolóxicos teñen erros, faltas e medidas absurdas, aínda despois de pasar o filtro de validación. Tocaba decidir como tratalos.

Rangos físicos. Para cada unha das 12 variables fixamos un intervalo razoable e descartamos calquera medida que caese fóra. Os límites saíron de **sentido común e un pouco de Wikipedia** para os casos dubidosos:

Variable	Rango aceptado
Temperatura (media/máx/mín)	-15 a 50 ℃
Humidade relativa	0 a 100 %
Precipitación diaria	0 a 500 L/m²
Vento medio / racha	0 a 50 / 0 a 80 m/s
Irradiación global	0 a 4 000
Horas de sol	0 a 18
Punto de rocío	-25 a 40 ℃
Presión atmosférica	750 a 1 080 hPa

A presión ten un límite inferior baixo (750 hPa) precisamente porque dúas das nosas estacións están **por riba dos 1 600 m de altitude**: Lardeira en Carballeda de Valdeorras (1 620 m) e Xares en A Veiga (1 762 m). A presión normal para esas alturas é xustamente arredor de 800 hPa, así que un rango estritamente «a nivel do mar» tería descartado todos os seus datos.

O resultado sorprendeunos pola positiva: **só 7 outliers en 1,1 millóns de medidas**. Practicamente todos eran 7 valores de humidade relativa máxima en As Petarelas (entre 101 % e 105 %), claros erros de calibración do sensor.

Interpolación de ocos illados. Quedaban días soltos sen dato. Dentro de cada estación, se entre dous días con dato había **un único día baleiro**, recheámolo coa media dos dous lados (interpolación lineal). Limitamos a 2 días consecutivos como máximo: máis aló diso, non é interpolar, é inventar.

Total recheado: **7 684 ocos**. Despois desta limpeza o dataset meteorolóxico estaba listo para cruzar cos incendios.

C.7 Paso 4 · Cruzar dous mundos: o problema das coordenadas

segunda semana de abril de 2026

Para que cada incendio quedase asociado coas súas condicións meteorolóxicas, había que **emparellar dous conxuntos de datos** que viñan en sistemas de coordenadas distintos:

- Os incendios de Civio veñen en **latitude/lonxitude WGS84** (o sistema GPS habitual).
- As estacións de MeteoGalicia veñen en **UTM zona 29N**, en metros desde o meridiano central (-9°), que é o sistema cartográfico oficial para Galicia.

Calcular distancias en lat/lon esixe a fórmula de Haversine (esfera). En UTM, en troques, basta a fórmula clásica de Pitágoras porque xa son metros nun plano. **Decidimos pasar todo a UTM** para simplificar.

Por que entender a fórmula. Existen librerías Python que fan a conversión nunha liña. Pero pedímoslle a Claude (a nosa IA executora, ver sección anterior) que implementase a fórmula exacta de proxección transversal de Mercator (Snyder, 1987) **desde cero**, sen depender de librerías externas. As **decisións** foron nosas (cal usar, por que, que rango de validez ten); a **execución do código** delegámola na IA. Tardamos máis ca cunha librería, pero entendemos exactamente o que estaba pasando, e o noso titor de matemáticas (Carlos) puido revisar a derivación.

Asignar estación a cada incendio. Para cada un dos 7 719 incendios, calculamos a distancia euclidiana ás 20 estacións e asignámoslle a máis próxima. Aí apareceu o **primeiro contratempo**: 1 933 incendios quedaron sen datos meteorolóxicos porque a súa estación

máis próxima non operaba ese día (avaría, mantemento, ou simplemente que a serie temporal aínda non chegaba).

A solución — respaldo espacial. En vez de descartar eses incendios, programamos unha estratexia de respaldo: ordenar **todas as estacións** por distancia e ir probando ata atopar unha que tivese dato para esa data. Se a 1ª non, a 2ª; se a 2ª non, a 3ª; e así sucesivamente.

Para o vento. Engadimos un segundo respaldo específico: nas 4 estacións sen anemómetro, recollemos as demais variables da estación principal pero o vento da seguinte estación máis próxima que tivese vento. 1 898 incendios usaron vento dunha estación alternativa.

Co respaldo espacial conseguimos asignar **datos meteorolóxicos a todos os incendios**. A distancia mediana á estación asignada quedou en **10,8 km** (o respaldo súbea un pouco respecto á asignación inxenua). Tomamos nota do número para a discusión final.

> **Nota retrospectiva** — As cifras deste paso e do seguinte (4 806 positivos, 92 844 filas, 10,8 km de mediana) corresponden á primeira versión do dataset. No Paso 9 atopamos un bug no filtro de calidade que, ao corrixilo, descartou 1 191 incendios sen meteoroloxía completa do día e endureceu o resultado a **4 512 positivos**, **93 221 filas** e **10,1 km** de mediana. A táboa comparativa antes/despois aparece máis abaixo, no Paso 9.

C.8 Paso 5 · Inventando os «non incendios»

terceira semana de abril de 2026

Tras a fusión, tiñamos 7 719 pares **(incendio, meteoroloxía)**. Pero un modelo de clasificación binaria precisa dúas clases: días con incendio **e** días sen incendio. Os negativos había que xeralos.

Tres opcións sobre a mesa:

- **Opción A** — mostrear aleatoriamente N días sen incendio, ratio 1:1 ou 1:2 cos positivos. Vantaxe: dataset balanceado, fácil de adestrar. Inconveniente: perdemos a inmensa maioría dos días-estación, e o modelo non aprende a distribución real do problema.
- **Opción B** — usar **todos** os días-estación dispoñibles (uns 107 000) como negativos, agás os 4 806 pares (data, estación) que coinciden con incendio. Vantaxe: realista e completo. Inconveniente: ratio 18:1, o que require manexar o desbalanceo despois.
- **Opción B+** — como B, pero engadindo unha **ventá de exclusión de ±3 días** arredor de cada incendio. Eses días nin se etiquetan como positivos nin como negativos: elimínámolos do dataset.

A decisión foi rápida e sen debate: **B+**. Era a opción que mellor reflectía o problema real, e a ventá de exclusión solucionaba un problema obvio. Se hai un incendio o 14 de agosto, o 13 e o 15 son meteorolóxicamente case idénticos, pero etiquetalos como «sen incendio» é enganoso (puido haber un incendio próximo non rexistrado, ou as condicións estaban no fío). Marcar eses días como ambiguos é máis honesto que forzalos a ser negativos.

O desbalanceo manexariámolo despois, na fase de modelado, con `class_weight='balanced'` e optimización do limiar de decisión. Investigámolo previamente: o estudo de **Zha et al. (2024) «Balancing the Scales: A Comprehensive Study on Class Imbalance»** (9 000 experimentos sobre 30 datasets) concluíu que esta estratexia é máis efectiva ca técnicas como SMOTE.

Dez variables derivadas. As 12 variables meteorolóxicas crudas (temperatura, humidade, vento, etc.) describen o día concreto pero non a **tendencia**. Engadimos 10 variables que capturan o contexto temporal:

- **Seca**: días consecutivos sen choiva, e precipitación acumulada en 7, 14 e 30 días.
- **Tendencias**: temperatura media e máxima dos últimos 3 días, humidade mínima dos últimos 3 días.
- **Día concreto**: amplitude térmica (T máxima menos mínima), mes do ano e día do ano.

A inspiración veu do **sistema FWI canadense** (no que se basea o IRDI que publica a Xunta), que dálle un papel central á humidade do combustible —é dicir, ao acumulado de seca— máis que ao día illado.

Definición formal. Para que quede claro o que é cada fila do dataset, sexa E o conxunto das 20 estacións e D o conxunto de días do período 2005–2021. Cada fila é un par $(d, e) \in D \times E$, etiquetado así:

- **Positivo** ($y = 1$): par (d, e) con incendio rexistrado na estación e na data d.
- **Excluído** (non entra no dataset): par (d, e) sen incendio, pero $|d - d_i| \leq 3$ días para algún incendio d_i na mesma estación.
- **Negativo** ($y = 0$): par (d, e) sen incendio e fóra de tódalas ventás de exclusión.

O algoritmo en pseudocódigo, simplificado:

```
fire_pairs = {(data_i, estacion_i) para cada incendio i}
exclusion_pairs = {(d±k, e) para (d,e) en fire_pairs, k en {-3..+3}}
exclusion_only = exclusion_pairs - fire_pairs
```

```
para cada fila (data, estacion) en meteo:
    se (data, estacion) en fire_pairs:    y = +1   # positivo
    senón se en exclusion_only:          y = -1   # descartar
    senón:                                y =  0   # negativo

# Importante: as features rolling (PP_acum_7d/14d/30d, dias_sin_lluvia,
# calcúlanse SOBRE A SERIE COMPLETA antes de filtrar os excluídos,
# para que non teñan ocos artificiais.
```

Resultado primeira execución: 4 806 positivos + 88 038 negativos = **92 844 filas**, con ratio 18,3:1. Quedaba listo para entrenar.

C.9 Paso 6 · Que modelos probar

finais de abril de 2026

Co dataset construído (92 844 filas, 22 variables), tocaba decidir **que algoritmos de aprendizaxe automática** usar. Decidimos probar **tres modelos**, do máis simple ao máis complexo: **Logistic Regression**, **Random Forest** e **XGBoost**. (Como contaremos máis adiante, ao final só executamos LR e RF: XGBoost quedou descartado por priorización temporal e por considerar que a comparativa lineal vs non-lineal con LR vs RF xa demostraba o que precisabamos.)

Por que non redes neuronais. Aquí non houbo investigación propia: foi **Carlos**, o noso titor, quen nos orientou. Recomendounos dous traballos clave para fundamentar a decisión sobre que tipo de modelo escoller en datos tabulares (números nunha táboa, non imaxes nin texto):

- **Grinsztajn et al. (2022) «Why do tree-based models still outperform deep learning on tabular data?»** (NeurIPS 2022).
- **Shwartz-Ziv Armon (2022) «Tabular data: Deep learning is not all you need»** (Information Fusion).

Ambos coinciden en que os modelos baseados en árbores son sistemáticamente superiores ao deep learning neste tipo de datos. Lemos os abstracts e as conclusións para entender o argumento, e adoptámolo. Polo tanto, descartamos redes neuronais, transformers e LLMs: están deseñados para datos non estruturados (imaxes, texto, audio), non para táboas numéricas como a nosa.

Outros descartes. Tamén deixamos fóra:

- **KNN** — escala mal con 100 000 filas (ten que calcular distancias a todos os puntos en cada predición).
- **SVM** — moi lento en datasets grandes (complexidade $O(n^2)$) e sen vantaxe clara fronte a XGBoost.

Estratexia: do simple ao complexo. A idea era empezar polo modelo máis sinxelo (Logistic Regression) e só pasar a un máis complexo se o anterior non era suficiente. Esta filosofía ten dúas vantaxes: primeira, evita complicar o proxecto sen necesidade; segunda, dá unha **liña base** clara contra a que comparar calquera modelo posterior. Se o Random Forest mellora moito á Logistic Regression, é porque as relacións entre meteoroloxía e incendios non son lineais. Se non mellora, non pagaba a pena complicar.

Esta estratexia tamén impresiona ao tribunal: non se elixe un modelo «porque si», xustifícase con datos comparativos.

A decisión de cales 3 modelos foi rápida e sen debate. A discusión real viría despois, ao ver os resultados.

C.10 Paso 7 · O modelo de referencia: Logistic Regression

14 de abril de 2026

Co dataset listo, executamos o noso primeiro modelo: **Logistic Regression**, tal e como o tiñamos planificado. É o modelo lineal máis simple razoable, e a súa función é darnos un **número de referencia**: se LR xa fai bo traballo, non precisamos modelos máis complexos; se non, calquera mellora posterior compárase contra ese número.

Configuración mínima: `class_weight='balanced'` para que o modelo non ignore a clase minoritaria (incendios, só o 5% dos casos), `solver='lbfgs'`, `max_iter=1000`. Sen sobre-ingeñería: queriamos ver canto daba un modelo «por defecto».

Resultados con limiar por defecto (0,5): F1 = **0,10** · Recall = 77% · Precision = 5,4% · PR-AUC = **0,0855**.

Á primeira lectura, o recall do 77% pareceunos bo: o modelo detecta máis de tres cuartos dos incendios. Pero o resto das cifras mátrano: emite **4 802 falsas alarmas** sobre **355 incendios reais** no test, é dicir, de cada 100 alertas, só 5 son certas. O modelo «grita lume» constantemente.

Optimización do limiar. Buscamos o limiar que maximizase o F1 e atopámolo en **0,746** (`TunedThresholdClassifierCV`). Con ese limiar máis esixente, as falsas alarmas baixan a 1 686, pero o recall cae ao 47%. F1 sobe a 0,15. Algo curioso: en problemas desbalanceados, o limiar óptimo adoita ser **menor** ca 0,5 (porque a clase positiva é rara). Que

aquí saíse maior é unha pista de que o modelo está moi pouco seguro das súas predicións positivas.

Os coeficientes desvelaron algo importante. Ao mirar que peso lle daba LR a cada variable, atopamos contradicións sospeitosas: a temperatura saía con coeficiente **negativo** (a maior temperatura, *menor* risco predito), o cal é o oposto do que ten sentido físicamente. A explicación máis probable é que un modelo lineal non capta interaccións: a temperatura por si soa non é decisiva, pero «temperatura alta + humidade baixa + 30 días sen choiva» si o é. LR non pode representar esa combinación.

Conclusión. Era o resultado esperable para un modelo de referencia: rendemento modesto pero útil. Os coeficientes contradictorios non eran un erro, eran a evidencia de que precisabamos un modelo capaz de aprender **non linealidades e interaccións**. Tocaba pasar ao seguinte: **Random Forest**.

C.11 Paso 8 · Random Forest: o salto cualitativo

28 de abril de 2026

Tras o resultado modesto e os coeficientes contradictorios de Logistic Regression, executamos o segundo modelo: **Random Forest**. A diferenza esencial é que un bosque de árbores pode aprender **interaccións** —ese «temperatura alta + humidade baixa + 30 días sen choiva» que LR non podía representar.

Configuración: 300 árbores (`n_estimators=300`), profundidade máxima 15 niveis (`max_depth=15`), mínimo 10 mostras por folla (`min_samples_leaf=10`), `class_weight='bal`
para o desbalanceo. Os límites á complexidade dos árbores (15 niveis, 10 mostras por folla) son protección contra **overfitting**: sen eles, cada árbore podería memorizar o conxunto de adestramento e fallar no test.

Resultados con limiar por defecto (0,5): F1 = **0,17** · Recall = 55% · Precision = 10% · PR-AUC = **0,1406**.

Xa só con isto, o salto respecto a LR é claro: o PR-AUC pasa de 0,0855 a 0,1406 (**+64%**) e a precision case se duplica (de 5,4% a 10%). O recall baixa un pouco (de 77% a 55%), pero o equilibrio xeral mellora.

Optimización do limiar. O óptimo saíu en **0,591** (lixeiramente superior a 0,5). F1 sobe a 0,187, precision a 11,9%. Comparativa final fronte a LR (ambos con limiar optimizado):

Métrica	Logistic Regression	Random Forest	Mellora
PR-AUC	0,0855	0,1406	**+64,4%**
F1 (tuned)	0,1521	0,1870	+22,9%
Precision	0,0906	0,1192	+31,6%

Importancia das variables: o que aprende o modelo. Esta é, para nós, a parte máis importante deste paso. A diferenza de LR (cuxos coeficientes saían contradictorios), o Random Forest devolveu un ranking de variables **coherente coa ciencia forestal**:

Rank	Variable	Importancia
1	días_sin_lluvia	0,117
2	PP_acum_7d (choiva 7 días)	0,112
3	HR_MIN_3d (humidade mínima 3 días)	0,091
4	PP_acum_14d	0,080
5	PP_acum_30d	0,069
6	HR_AVG (humidade media)	0,067

As **6 variables máis predictivas son indicadores de seca e humidade**. Isto coincide exactamente co deseño do **sistema FWI canadense**, base do IRDI publicado pola Xunta: o factor primario é a humidade do combustible, é dicir, canto leva sen chover e canta auga ten o aire. A temperatura, que intuitivamente parecería decisiva, queda en posicións intermedias.

Conclusión. O modelo non é perfecto (F1 = 0,19 segue sendo modesto), pero a mellora respecto ao modelo de referencia e a coherencia da importancia das variables coa literatura científica son a confirmación que precisabamos: as relacións entre meteoroloxía e incendios non son lineais, e Random Forest captúraas. Decidimos non avanzar a XGBoost: a comparativa lineal vs non lineal xa estaba feita.

C.12 Paso 9 · Auditoría e correccións de últimos días

9 de maio de 2026 — a 6 días da defensa

A unha semana da defensa, antes de pulir a memoria, decidimos facer **unha revisión completa do código**. Pasámoslle todo o repositorio a un axente de IA cunha consigna sinxela: «busca bugs, incoherencias, hardcodes e decisións metodolóxicas frouxas». A revisión devolveu **tres problemas reais** que ata ese momento non viramos.

Bug 1 (alto): a curva PR de comparación era mentireira. O gráfico que enfrontaba LR contra Random Forest debuxaba LR como unha **liña horizontal** na altura do seu PR-AUC, no canto da súa curva real. Visualmente facía pensar que LR era un clasificador trivial, cando en realidade tiña a súa propia forma. Era un erro de visualización, non de cálculo, pero ao mostrar este gráfico nunha defensa daría unha imaxe equivocada.

Bug 2 (medio): métricas hardcodeadas. No script que adestraba Random Forest, as cifras de Logistic Regression aparecían como números fixos no código (`lr_pr_auc = 0.0855`, `lr_f1 = 0.1521`...). Se algún día se reentrenaba LR, eses números quedarían desactualizados sen aviso. Era un risco de reproducibilidade.

Bug 3 (medio, o máis interesante): a heurística de fusión era demasiado laxa. Cando asignamos cada incendio á súa estación meteorolóxica máis próxima (Paso 4), o criterio de «esta estación ten datos para esta data» mirábase só sobre a temperatura. Bastaba que houbese **un único valor** —a temperatura— para considerar que esa estación servía. Pero podía non ter humidade nin precipitación, e o modelo recibíallo igualmente con NaN.

A decisión metodolóxica. Cambiamos o criterio: agora unha estación só conta como «con datos» se ten as **tres variables base presentes simultaneamente** (temperatura, humidade e precipitación). É unha regra máis estrita pero máis honesta. A consecuencia: descartámonos **1 191 incendios (15,4 %)** porque ningunha das 20 estacións dispuña das tres variables completas para a súa data exacta. A cambio, garántese que os **6 528 incendios procesados (84,6 %)** non conteñen valores ausentes nas variables máis informativas.

Considerámolo un avance metodolóxico, non unha perda. Mellor un dataset máis pequeno e íntegro que un máis grande con imputacións artificiais nas variables que despois deciden a etiqueta do exemplo.

Re-execución do pipeline e novos resultados. Tras corrixir os tres bugs e re-executar todo:

Métrica	Antes do fix	Despois do fix
Filas dataset	92 844	**93 221**
Positivos	4 806	**4 512**
Distancia mediana á estación	10,8 km	**10,1 km**
LR · PR-AUC	0,0855	**0,0883**
LR · F1 (tuned)	0,1521	**0,1593**
RF · PR-AUC	0,1406	**0,1485**
RF · F1 (tuned)	0,1870	**0,2082**
RF · limiar óptimo	0,591	**0,613**
Mellora RF vs LR	+64 %	**+68 %**

Ningunha métrica empeora. A auditoría confirmou unha lección importante: o método de traballo (revisar antes de presentar) é tan parte do proxecto como o modelo en si. Aprendemos que os bugs poden esconderse durante semanas se non se buscan activamente.

C.13 Paso 10 · LUMIA: a interface para mostrar o modelo en directo

primeira semana de maio de 2026

Tiñamos un modelo entrenado e gardado nun ficheiro `.pkl` de 43 MB. Funciona, pero **non se ve**: para usalo cómpre executar Python, cargar o pickle, pasarlle un vector de 22 números e interpretar a saída. Iso non vale para demostrar nada nunha defensa.

Decidimos construír unha pequena **interface web** que permitise:

1. Ver o **risco predito hoxe e nos próximos días** para cada estación de Ourense, usando datos meteorolóxicos en tempo real.
2. Explorar o **dataset histórico** (os 7 719 incendios de 2005-2021 e as 20 estacións) sobre un mapa interactivo.
3. **Probar o modelo manualmente** introducindo condicións meteorolóxicas inventadas, para ver como cambia o risco.

A bautizamos **LUMIA** —*o lume en Ourense*—. **As decisións sobre que mostrar e como organizalo foron nosas**: as tres pantallas, o sistema visual, a paleta de cores, a estrutura da navegación, os textos en galego. **A execución técnica delegámola en Claude**, que escribiu o código (FastAPI + Jinja2 + HTMX para a parte web, scikit-learn para cargar o modelo, MeteoGalicia API para os datos en tempo real, Docker para o empaquetado e Cloudflare Tunnel para o despregamento).

Está accesible publicamente en **https://lumia.cabanelas.info/**.

O propósito de LUMIA é ser unha **demostración funcional**, non un sistema de produción. Un agasallo para o tribunal: en vez de ler números nunha táboa da memoria, poden abrir a páxina nun móbil ou nun portátil e ver o modelo funcionando con datos reais de hoxe mesmo.

Os detalles técnicos da implementación (decisións de stack, arquitectura, despregamento) están documentados en dous ficheiros aparte do repositorio: `interfaz.md` e `despliegue.md`. Aquí no caderno só queriamos deixar constancia da decisión: **un modelo que non se pode tocar é difícil de defender**.

C.14 Paso 11 · A pregunta de Carlos: que predí exactamente o modelo?

11–12 de maio de 2026 — 3 días antes da defensa

A última semana antes da defensa, Carlos puxo o dedo nunha cousa que tiñamos enterrada en frases longas pero non destacada en ningún sitio: **"¿Que predí exactamente o voso modelo? Porque non é o mesmo 'haberá un incendio' que 'as condicións son favorables a que aconteza' "**.

A pregunta parece de matiz, pero non é. O modelo aprende **patróns meteorolóxicos** asociados a días con incendio. Non aprende **causas**. Cando o modelo asigna un risco alto, está dicindo: "as condicións son das que historicamente se asociaron con incendios". Pero **un incendio require unha ignición** —humana ou natural—, e iso non está no dataset.

A consecuencia é importantísima para interpretar os resultados:

- Un **falso positivo** (alto risco predito sen incendio rexistrado) **non é necesariamente un erro do modelo**. Pode reflectir simplemente que ese día non houbo ignición —ninguén tirou unha cabicha, ningún rayo caeu—. As condicións estaban, pero faltou o detonante.
- Un **falso negativo** (incendio con risco baixo predito) tampouco invalida o modelo: pode ser un incendio causado por circunstancias non meteorolóxicas (intencionado, neglixencia eléctrica, queima agrícola con condicións suaves).

Lembramos o dato do EGIF: **o 76 % dos incendios en Galicia son intencionados**. Polo tanto, ata o modelo perfecto non debería predicir todos correctamente: hai unha capa enteira (a humana) que a meteoroloxía non ve.

A discusión derivou en dous cambios concretos.

Primeiro, **na memoria**: cambiamos o resumo (galego, castelán e inglés) e engadimos un parágrafo no capítulo de discusión para deixar a distinción explícita. Antes diciamos "predición do risco de incendio"; agora dicimos **"predición das condicións meteorolóxicas favorables a incendio"**. Sete palabras máis, pero mudan completamente o que se promete.

Segundo, **en LUMIA**: engadimos un *callout* destacado nas catro pantallas (Sobre, Ourense hoxe, Casos reais, Predictor) coa mesma aclaración, e cambiamos o texto do resultado do predictor: onde antes saía "**INCENDIO probable**" agora sae "**Condicións favorables a incendio**". Carlos mencionou específicamente o falso positivo de Baltar dos Casos reais como exemplo: alto risco predito, sen incendio rexistrado. Antes parecía un fallo; agora explícase como o que é —un día propicio sen ignición.

Aprendizaxe. As preguntas dos titores son **tan parte do método como o código**. Non foi un erro do modelo nin do dataset: foi un erro da nosa **redacción**. E iso só se ve cando alguén que sabe pregunta.

C.15 Paso 12 · Cerre: o que aprendemos, o que queda

15 de maio de 2026 — defensa

Pechamos este caderno coa defensa pública. Antes de gardalo, queriamos deixar tres cousas escritas: o que aprendemos, o que queda pendente, e o que faríamos doutro xeito se empezasemos hoxe.

C.15.1 Que aprendemos

Sobre datos. Que construír un dataset propio non se fai nunha tarde. Foron varias semanas de descargas, limpeza, decisión de criterios, conversión de coordenadas e estratexias de respaldo. O traballo *invisible* (o que non aparece nun gráfico final) é a maior parte do proxecto.

Sobre modelos. Que un resultado modesto, ben explicado, vale máis que un resultado inflado. O noso F1 = 0,21 non é espectacular, pero entendemos por que é así (conxunto de test difícil, ausencia do factor humano que causa o 76 % dos incendios) e podemos defendelo. Aprendemos tamén que un modelo de referencia é imprescindible: sen Logistic Regression non poderiamos demostrar que Random Forest aporta algo real.

Sobre método. Que **revisar antes de presentar** salva proxectos. A auditoría do Paso 9 atopou tres bugs que levaban semanas escondidos. Sen esa revisión, defenderiamos cifras menos honestas. O método é tan parte do proxecto como o resultado.

Sobre IA. Que usar Claude como executor técnico —pero mantendo nós as decisións— é unha forma honesta e produtiva de traballar. A IA non substitúe o criterio: ampliao. Se non sabes que pedirlle, ou se non sabes ler o que devolve, a IA non te axuda.

Sobre a precisión das preguntas. Que distinguir "predición de incendio" de "predición de condicións favorables a incendio" (Paso 11) non é un matiz: cambia como se interpretan os falsos positivos, os falsos negativos e ata como se redacta o resumo. Aprendemos que **as palabras importan tanto como os algoritmos**.

C.15.2 O que queda pendente

- **XGBoost.** Planificámolo no Paso 6 e descartámolo por tempo. Non sabemos se aportaría máis ca Random Forest; sería o seguinte experimento natural.
- **Factor humano.** O 76 % dos incendios en Galicia son intencionados, segundo os datos do EGIF. O noso modelo non recolle nada diso (queimas agrícolas, neglixencias, incendiarismo). É a maior limitación científica.
- **Ampliación xeográfica.** Galicia ten 155 estacións meteorolóxicas; nós usamos 20 de Ourense. Estender o dataset a toda Galicia é unha extensión natural.
- **Variables non meteorolóxicas.** Tipo e estado da vexetación, topografía, accesibilidade, distancia a poboación. Combinadas coa meteoroloxía darían un modelo máis completo.
- **Datos en tempo real.** LUMIA xa consume previsións actuais, pero o modelo non se reentrena con datos novos. Un sistema de produción necesitaría isto.

C.15.3 O que faríamos diferente

Auditar antes. Os bugs do Paso 9 levaban semanas no código. Se tivésemos posto unha revisión periódica desde o principio, atopariámolos antes.

Empezar pola arquitectura final. LUMIA empezou en Streamlit e despois migrámolo a FastAPI. O Streamlit foi útil para validar a idea, pero perdeu tempo que se podía ter aforrado.

Documentar a medida que se decide. Algunhas decisións (rangos físicos, criterios de fusión) ficaron documentadas só *despois* de tomarse. Decidimos sobre a marcha e logo escribimos por que. Mellor decidir e escribir á vez.

C.15.4 Agradecementos

A **Carlos Ferreiro García**, titor no IES As Lagoas, por orientarnos no enfoque e na bibliografía científica. A **José Ángel Cid Arújo**, cotitor universitario na UVigo, polo seguimento.

E a **MeteoGalicia** e **Civio**, pola apertura dos datos públicos sen a cal este proxecto non sería posible.

Caderno pechado a 15 de maio de 2026.

Apéndice D

Código fonte

Este apéndice recolle os scripts máis relevantes do pipeline do proxecto: a fusión espacial-temporal con conversión de coordenadas, a construción do dataset de clasificación, o adestramento dos dous modelos (regresión loxística e Random Forest) e a exportación do modelo final para a interfaz LUMIA. Os scripts ETL auxiliares (descarga da API, limpeza de datos brutos) cínxense a operacións estándar de pandas e *requests*, polo que se omiten neste apéndice; o seu funcionamento describíse en linguaxe natural no caderno de campo (apéndice C) e o código fonte completo está dispoñible no repositorio do proxecto.

Antes de cada listado descríbese brevemente *que fai* o script e cales son os *comandos principais*.

D.1 `fusionar_datasets.py`

Fusiona os incendios cos datos meteorolóxicos en dúas fases. **Primeira**: converte as coordenadas xeográficas de cada incendio (latitude/lonxitude, WGS84) a coordenadas UTM zona 29N (en metros). **Segunda**: para cada incendio busca a estación meteorolóxica máis próxima con datos para esa data, aplicando unha estratexia de respaldo se a primeira non está dispoñible. Trata á parte as 4 estacións sen anemómetro buscando o vento na seguinte estación máis próxima.

Comandos principais:

- `latlon_to_utm29n(lat, lon)`: implementación dende cero das fórmulas oficiais de **(author?)** [10] para a proxección transversal de Mercator sobre o elipsoide WGS84. Decidimos non depender de `pyproj` para entender en detalle o cálculo.
- `dist_utm(x1, y1, x2, y2)`: distancia euclidiana en metros entre dous puntos UTM (Pitágoras).

- `fire_station_ranks`: para cada incendio, lista das 20 estacións ordenadas por distancia ascendente.
- **Bucle de asignación con respaldo**: percorre o ranking ata atopar a primeira estación con datos meteorolóxicos válidos para esa data. Se a máis próxima non ten dato, proba a 2.ª, a 3.ª, etc.
- **Asignación específica do vento**: para as 4 estacións sen anemómetro, o vento recóllese da seguinte estación máis próxima que si o mida.

```python
#!/usr/bin/env python3
"""
Paso 8.4-8.7 — Fusion espacial-temporal de incendios + meteorologia.

1. Convierte coordenadas de incendios de lat/lon WGS84 a UTM zona 29N
2. Para cada incendio, ordena TODAS las estaciones por distancia
3. Busca la estacion mas cercana que tenga datos para esa fecha
4. Para variables de viento, busca la estacion mas cercana CON
    ↪ anemometro
5. Guarda el dataset fusionado

Uso: python3 src/fusionar_datasets.py
"""

import pandas as pd
import numpy as np
import math
import os

# Paths
PROJECT_DIR = os.path.dirname(os.path.dirname(__file__))
PROCESSED_DIR = os.path.join(PROJECT_DIR, "data", "processed")
INC_CSV = os.path.join(PROCESSED_DIR, "incendios_ourense.csv")
METEO_CSV = os.path.join(PROCESSED_DIR, "meteo_ourense.csv")
ESTACIONES_CSV = os.path.join(PROCESSED_DIR, "estaciones_ourense.csv")
OUTPUT_CSV = os.path.join(PROCESSED_DIR, "dataset_fusionado.csv")

# Estaciones que NO miden viento (identificadas en paso 7)
SIN_VIENTO = {10111, 10113, 10114, 10148}

# Variables de viento (se obtienen de estacion alternativa si la
    ↪ primaria no tiene)
VARS_VIENTO = ["VV_AVG_10m", "VV_MAX_10m"]

# All meteorological variables
```

```
METEO_VARS = [
    "TA_AVG_1.5m", "TA_MAX_1.5m", "TA_MIN_1.5m",
    "HR_AVG_1.5m", "HR_MIN_1.5m",
    "PP_SUM_1.5m",
    "VV_AVG_10m", "VV_MAX_10m",
    "IRD_SUM_1.5m", "HSOL_SUM_1.5m",
    "TO_AVG_1.5m", "PR_AVG_1.5m",
]


def latlon_to_utm29n(lat, lon):
    """
    Convert WGS84 lat/lon to UTM zone 29N coordinates.
    UTM zone 29N: central meridian = -9 degrees.
    Uses the exact transverse Mercator formulas (Snyder, 1987).
    """
    lat_rad = math.radians(lat)
    lon_rad = math.radians(lon)
    lon0_rad = math.radians(-9)

    a = 6378137.0
    f = 1 / 298.257223563
    e2 = 2 * f - f * f
    e_prime2 = e2 / (1 - e2)

    N = a / math.sqrt(1 - e2 * math.sin(lat_rad) ** 2)
    T = math.tan(lat_rad) ** 2
    C = e_prime2 * math.cos(lat_rad) ** 2
    A_ = (lon_rad - lon0_rad) * math.cos(lat_rad)

    M = a * (
        (1 - e2 / 4 - 3 * e2**2 / 64 - 5 * e2**3 / 256) * lat_rad
        - (3 * e2 / 8 + 3 * e2**2 / 32 + 45 * e2**3 / 1024) * math.sin(2
    ↪   * lat_rad)
        + (15 * e2**2 / 256 + 45 * e2**3 / 1024) * math.sin(4 * lat_rad)
        - (35 * e2**3 / 3072) * math.sin(6 * lat_rad)
    )

    k0 = 0.9996
    easting = k0 * N * (
        A_ + (1 - T + C) * A_**3 / 6
        + (5 - 18 * T + T**2 + 72 * C - 58 * e_prime2) * A_**5 / 120
    ) + 500000

    northing = k0 * (
        M + N * math.tan(lat_rad) * (
```

```
            A_**2 / 2
            + (5 - T + 9 * C + 4 * C**2) * A_**4 / 24
            + (61 - 58 * T + T**2 + 600 * C - 330 * e_prime2) * A_**6 /
    ↪ 720
        )
    )
    return easting, northing


def dist_utm(x1, y1, x2, y2):
    """Euclidean distance in meters between two UTM points."""
    return math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)


def main():
    print("=" * 70)
    print("FUSION ESPACIAL-TEMPORAL: INCENDIOS + METEOROLOGIA")
    print("=" * 70)

    # --- Load data ---
    inc = pd.read_csv(INC_CSV, parse_dates=["fecha"])
    meteo = pd.read_csv(METEO_CSV, parse_dates=["fecha"])
    estaciones = pd.read_csv(ESTACIONES_CSV)

    print(f"\nDatos cargados:")
    print(f"  Incendios: {len(inc):,} registros")
    print(f"  Meteorologia: {len(meteo):,} filas (dias x estaciones)")
    print(f"  Estaciones: {len(estaciones)}")

    # --- STEP 1: Convert fire coordinates to UTM ---
    print(f"\n--- PASO 1: Conversion de coordenadas a UTM zona 29N ---")
    utm_coords = inc.apply(lambda r: latlon_to_utm29n(r.lat, r.lng),
    ↪ axis=1)
    inc["utmx"] = [c[0] for c in utm_coords]
    inc["utmy"] = [c[1] for c in utm_coords]
    print(f"  {len(inc):,} puntos convertidos")

    # --- STEP 2: Compute distance from each fire to ALL stations ---
    print(f"\n--- PASO 2: Calculo de distancias a todas las estaciones
    ↪ ---")
    est_coords = {
        int(row.estacion_id): (row.utmx, row.utmy)
        for _, row in estaciones.iterrows()
    }

    # For each fire, rank all stations by distance
```

```
    fire_station_ranks = []
    for _, fire in inc.iterrows():
        fx, fy = fire.utmx, fire.utmy
        dists = []
        for eid, (ex, ey) in est_coords.items():
            d = dist_utm(fx, fy, ex, ey)
            dists.append((eid, d))
        dists.sort(key=lambda x: x[1])
        fire_station_ranks.append(dists)

    print(f"  Ranking calculado para {len(fire_station_ranks):,} incendios x {len(est_coords)} estaciones")

    # --- STEP 3: Build index of available meteo data by (date, station) ---
    print(f"\n--- PASO 3: Indice de disponibilidad meteorologica ---")
    meteo["fecha_str"] = meteo["fecha"].dt.strftime("%Y-%m-%d")

    # "Tiene datos utiles" = tiene las 3 variables base presentes en TODAS las estaciones:
    # temperatura, humedad y precipitacion. El viento se trata aparte porque 4 estaciones
    # no tienen anemometro (10111, 10113, 10114, 10148) y nunca tendran ese dato.
    BASIC_VARS = ["TA_AVG_1.5m", "HR_AVG_1.5m", "PP_SUM_1.5m"]

    # Vectorizado: una sola pasada en lugar de 3 iterrows sobre 106k filas
    meteo["estacion_id_int"] = meteo["estacion_id"].astype(int)
    has_basics = meteo[BASIC_VARS].notna().all(axis=1)
    has_wind = meteo["VV_AVG_10m"].notna()

    meteo_available = set(zip(meteo.loc[has_basics, "fecha_str"],
                              meteo.loc[has_basics, "estacion_id_int"]))
    meteo_wind_available = set(zip(meteo.loc[has_wind, "fecha_str"],
                                   meteo.loc[has_wind, "estacion_id_int"]))

    # Index meteo como dict: (date_str, station_id) -> row dict
    meteo_index = {
        (row["fecha_str"], row["estacion_id_int"]): row
        for _, row in meteo.iterrows()
    }

    print(f"  Pares (fecha, estacion) con datos basicos (T+HR+PP): {len(meteo_available):,}")
```

```
    print(f"  Pares (fecha, estacion) con viento:                    {len(
    ↪ meteo_wind_available):,}")

    # --- STEP 4: For each fire, find nearest station WITH data for that
    ↪  date ---
    print(f"\n--- PASO 4: Asignacion con fallback ---")
    print(f"  Estrategia: si la estacion mas cercana no tiene datos para
    ↪  esa fecha,")
    print(f"  se prueba la siguiente mas cercana, y asi sucesivamente.")

    results = []
    stats = {
        "matched_1st": 0, "matched_2nd": 0, "matched_3rd_plus": 0,
        "no_match": 0, "wind_from_alt": 0,
    }

    for i, (_, fire) in enumerate(inc.iterrows()):
        fecha_str = fire.fecha.strftime("%Y-%m-%d")
        ranks = fire_station_ranks[i]

        # Find nearest station with data for this date
        matched_est = None
        matched_dist = None
        matched_rank = None

        for rank_idx, (eid, dist) in enumerate(ranks):
            if (fecha_str, eid) in meteo_available:
                matched_est = eid
                matched_dist = dist
                matched_rank = rank_idx
                break

        if matched_est is None:
            stats["no_match"] += 1
            row = {
                "id": fire.id, "fecha": fire.fecha, "lat": fire.lat, "
    ↪ lng": fire.lng,
                "municipio": fire.municipio, "superficie": fire.
    ↪ superficie,
                "causa": fire.causa, "causa_supuesta": fire.
    ↪ causa_supuesta,
                "estacion_id": None, "dist_estacion_m": None,
                "estacion_rank": None,
                "estacion_viento_id": None, "dist_viento_m": None,
            }
            for var in METEO_VARS:
```

```
                row[var] = None
            results.append(row)
            continue

        if matched_rank == 0:
            stats["matched_1st"] += 1
        elif matched_rank == 1:
            stats["matched_2nd"] += 1
        else:
            stats["matched_3rd_plus"] += 1

        # Get meteo data from matched station
        meteo_row = meteo_index[(fecha_str, matched_est)]

        row = {
            "id": fire.id, "fecha": fire.fecha, "lat": fire.lat, "lng":
    ↪ fire.lng,
            "municipio": fire.municipio, "superficie": fire.superficie,
            "causa": fire.causa, "causa_supuesta": fire.causa_supuesta,
            "estacion_id": matched_est, "dist_estacion_m": matched_dist,
            "estacion_rank": matched_rank + 1,  # 1-based
            "estacion_viento_id": matched_est, "dist_viento_m":
    ↪ matched_dist,
        }

        for var in METEO_VARS:
            row[var] = meteo_row.get(var)

        # If matched station has no wind, find nearest station WITH wind
    ↪  for this date
        if matched_est in SIN_VIENTO or pd.isna(row.get("VV_AVG_10m")):
            for eid, dist in ranks:
                if (fecha_str, eid) in meteo_wind_available:
                    wind_row = meteo_index[(fecha_str, eid)]
                    row["VV_AVG_10m"] = wind_row.get("VV_AVG_10m")
                    row["VV_MAX_10m"] = wind_row.get("VV_MAX_10m")
                    row["estacion_viento_id"] = eid
                    row["dist_viento_m"] = dist
                    stats["wind_from_alt"] += 1
                    break

        results.append(row)

    result_df = pd.DataFrame(results)

    print(f"\n  Resultados de la asignacion:")
```

```
    print(f"    Asignados a 1a estacion mas cercana:  {stats['matched_1st']:,}")
    print(f"    Asignados a 2a estacion mas cercana:  {stats['matched_2nd']:,}")
    print(f"    Asignados a 3a+ estacion mas cercana: {stats['matched_3rd_plus']:,}")
    print(f"    Sin ningun dato meteorologico:        {stats['no_match']:,}")
    print(f"    Viento desde estacion alternativa:    {stats['wind_from_alt']:,}")

    total_matched = stats["matched_1st"] + stats["matched_2nd"] + stats["matched_3rd_plus"]
    print(f"\n    TOTAL CON DATOS: {total_matched:,} de {len(inc):,} ({total_matched/len(inc)*100:.1f}%)")

    # --- STEP 5: Final stats and save ---
    print(f"\n--- PASO 5: Estadisticas finales ---")
    print(f"  Registros: {len(result_df):,}")

    print(f"\n  Disponibilidad por variable:")
    for var in METEO_VARS:
        nn = result_df[var].notna().sum()
        print(f"    {var:<20} {nn:>6,} ({nn/len(result_df)*100:.1f}%)")

    # Distance stats for matched fires
    matched_df = result_df[result_df.estacion_id.notna()]
    print(f"\n  Distancia a estacion asignada (solo los {len(matched_df):,} con datos):")
    print(f"    Mediana: {matched_df.dist_estacion_m.median()/1000:.1f} km")
    print(f"    Media:   {matched_df.dist_estacion_m.mean()/1000:.1f} km")
    print(f"    P95:     {matched_df.dist_estacion_m.quantile(0.95)/1000:.1f} km")
    print(f"    Max:     {matched_df.dist_estacion_m.max()/1000:.1f} km")

    # Rank distribution
    print(f"\n  Distribucion del ranking de estacion usada:")
    for rank, count in result_df.estacion_rank.value_counts().sort_index().items():
        pct = count / len(matched_df) * 100
        label = "mas cercana" if rank == 1 else f"{int(rank)}a mas cercana"
        print(f"    Estacion {label}: {count:,} ({pct:.1f}%)")
```

```
275
276     # Save
277     print(f"\n--- GUARDADO ---")
278     result_df.to_csv(OUTPUT_CSV, index=False)
279     size_mb = os.path.getsize(OUTPUT_CSV) / (1024 * 1024)
280     print(f"  Fichero: {OUTPUT_CSV}")
281     print(f"  Tamano: {size_mb:.2f} MB")
282     print(f"  Filas: {len(result_df):,}, Columnas: {len(result_df.
    ↪ columns)}")
283
284
285 if __name__ == "__main__":
286     main()
```

D.2 `construir_dataset_clasificacion.py`

Crea o dataset final de clasificación a partir do meteorolóxico e dos incendios fusionados. Etiqueta cada par (data, estación) como **positivo** (con incendio), **negativo** (sen incendio) ou **excluído** (a ± 3 días dun incendio na mesma estación). Calcula 10 variables derivadas (indicadores de seca, tendencias térmicas, humidade, calendario) e devolve `dataset_modelo.csv` listo para os modelos.

Comandos principais:

- `consecutive_dry_days(precip, threshold)`: conta días consecutivos con precipitación menor de 1 mm: produce a variable `dias_sin_lluvia`, a máis importante segundo Random Forest.
- `rolling(7).sum()`: precipitación acumulada en xanela móbil de 7 días. Aplícase tamén con xanelas de 14 e 30 días.
- `compute_derived_features(df)`: orquestra o cálculo das 10 variables derivadas para cada estación: indicadores de seca, tendencias térmicas, amplitude térmica e variables de calendario.
- **Construción da ventá de exclusión**: para cada par (data, estación) con incendio, marca os ± 3 días arredor (na mesma estación) como excluídos do dataset, para evitar que o modelo veña condicións case idénticas etiquetadas de xeito oposto.

Decisión metodolóxica: os cálculos de xanela móbil execútanse *antes* de filtrar as filas excluídas, sobre a serie temporal continua de cada estación. Filtrar primeiro produciría ocos artificiais nas medias móbiles.

```python
#!/usr/bin/env python3
"""
Paso 9 — Construccion del dataset de clasificacion binaria.

1. Carga meteo_ourense.csv (todos los dias-estacion) y dataset_fusionado.csv (incendios)
2. Etiqueta cada dia-estacion como incendio (1) o no-incendio (0)
3. Aplica ventana de exclusion ±3 dias alrededor de incendios
4. Calcula 10 features derivadas (sequia, tendencias, calendario)
5. Guarda dataset_modelo.csv listo para entrenar

Uso: python3 src/construir_dataset_clasificacion.py
"""

import pandas as pd
import numpy as np
import os

# Import from config
import sys
sys.path.insert(0, os.path.dirname(__file__))
from config import (
    PROCESSED_DIR, METEO_CSV, DATASET_FUSIONADO_CSV, ESTACIONES_CSV,
    METEO_VARIABLES, WIND_VARIABLES, STATIONS_WITHOUT_WIND,
)

OUTPUT_CSV = os.path.join(PROCESSED_DIR, "dataset_modelo.csv")

# Exclusion window: ±N days around each fire are neither positive nor negative
EXCLUSION_WINDOW_DAYS = 3

# Dry day threshold (mm)
DRY_THRESHOLD = 1.0


def consecutive_dry_days(precip_series, threshold=DRY_THRESHOLD):
    """Count consecutive days with precipitation below threshold."""
    result = np.zeros(len(precip_series), dtype=int)
    count = 0
    for i, val in enumerate(precip_series.values):
        if pd.notna(val) and val < threshold:
            count += 1
        else:
            count = 0
```

```
            result[i] = count
    return result


def compute_derived_features(df):
    """
    Compute 10 derived features for a single station's time series.
    DataFrame must be sorted by date with no duplicate dates.
    """
    # Drought indicators
    df["dias_sin_lluvia"] = consecutive_dry_days(df["PP_SUM_1.5m"])
    df["PP_acum_7d"] = df["PP_SUM_1.5m"].rolling(7, min_periods=4).sum()
    df["PP_acum_14d"] = df["PP_SUM_1.5m"].rolling(14, min_periods=7).sum
    ↪ ()
    df["PP_acum_30d"] = df["PP_SUM_1.5m"].rolling(30, min_periods=15).
    ↪ sum()

    # Temperature trends
    df["TA_AVG_3d"] = df["TA_AVG_1.5m"].rolling(3, min_periods=2).mean()
    df["TA_MAX_3d"] = df["TA_MAX_1.5m"].rolling(3, min_periods=2).max()

    # Humidity trend
    df["HR_MIN_3d"] = df["HR_MIN_1.5m"].rolling(3, min_periods=2).min()

    # Thermal amplitude
    df["amplitud_termica"] = df["TA_MAX_1.5m"] - df["TA_MIN_1.5m"]

    # Calendar features
    df["mes"] = df["fecha"].dt.month
    df["dia_del_ano"] = df["fecha"].dt.dayofyear

    return df


def main():
    print("=" * 70)
    print("CONSTRUCCION DEL DATASET DE CLASIFICACION")
    print("=" * 70)

    # --- LOAD DATA ---
    meteo = pd.read_csv(METEO_CSV, parse_dates=["fecha"])
    fusionado = pd.read_csv(DATASET_FUSIONADO_CSV, parse_dates=["fecha"
    ↪ ])
    estaciones = pd.read_csv(ESTACIONES_CSV)

    print(f"\nDatos cargados:")
```

```
    print(f"  Meteo: {len(meteo):,} filas")
    print(f"  Fusionado: {len(fusionado):,} incendios")
    print(f"  Estaciones: {len(estaciones)}")

    # --- STEP 1: Identify fire pairs (date, station) ---
    print(f"\n--- PASO 1: Identificar pares con incendio ---")

    fusionado["fecha_str"] = fusionado["fecha"].dt.strftime("%Y-%m-%d")
    fire_pairs = set()
    for _, row in fusionado.iterrows():
        if pd.notna(row.estacion_id):
            fire_pairs.add((row.fecha_str, int(row.estacion_id)))

    print(f"  Pares unicos (fecha, estacion) con incendio: {len(
    ↪ fire_pairs):,}")
    print(f"  ({len(fusionado):,} incendios -> {len(fire_pairs):,} pares
    ↪ , porque varios incendios "
          f"pueden caer en la misma estacion el mismo dia)")

    # --- STEP 2: Build exclusion window ---
    print(f"\n--- PASO 2: Ventana de exclusion ±{EXCLUSION_WINDOW_DAYS}
    ↪ dias ---")

    from datetime import timedelta
    exclusion_pairs = set()
    for fecha_str, est_id in fire_pairs:
        fecha = pd.Timestamp(fecha_str)
        for delta in range(-EXCLUSION_WINDOW_DAYS, EXCLUSION_WINDOW_DAYS
    ↪  + 1):
            excl_date = (fecha + timedelta(days=delta)).strftime("%Y-%m
    ↪ -%d")
            exclusion_pairs.add((excl_date, est_id))

    # Fire days themselves are NOT excluded (they're positives)
    exclusion_only = exclusion_pairs - fire_pairs
    print(f"  Pares en zona de exclusion: {len(exclusion_only):,}")

    # --- STEP 3: Label all meteo rows ---
    print(f"\n--- PASO 3: Etiquetar todas las filas ---")

    meteo["fecha_str"] = meteo["fecha"].dt.strftime("%Y-%m-%d")
    meteo["estacion_id_int"] = meteo["estacion_id"].astype(int)
    meteo["_pair"] = list(zip(meteo["fecha_str"], meteo["estacion_id_int
    ↪ "]))

    # Label: 1 = fire, 0 = no fire, -1 = exclusion zone
```

```
    labels = []
    for pair in meteo["_pair"]:
        if pair in fire_pairs:
            labels.append(1)
        elif pair in exclusion_only:
            labels.append(-1)
        else:
            labels.append(0)

    meteo["incendio"] = labels

    n_pos = (meteo["incendio"] == 1).sum()
    n_neg = (meteo["incendio"] == 0).sum()
    n_excl = (meteo["incendio"] == -1).sum()
    print(f"  Positivos (incendio=1):     {n_pos:,}")
    print(f"  Negativos (incendio=0):     {n_neg:,}")
    print(f"  Excluidos (ventana):        {n_excl:,}")

    # Remove excluded rows
    df = meteo[meteo["incendio"] >= 0].copy()
    print(f"  Filas tras exclusion:       {len(df):,}")
    print(f"  Ratio negativo:positivo:    {n_neg/n_pos:.1f}:1")

    # --- STEP 4: Compute derived features ---
    print(f"\n--- PASO 4: Features derivadas (por estacion) ---")

    # Las rolling features se calculan sobre la serie temporal continua
    # de cada estacion (sin huecos), y solo despues filtramos las filas
    # excluidas por la ventana ±3 dias. Si filtrasemos antes, los
    ↪ rolling
    # se calcularian sobre series con huecos y darian valores
    ↪ incorrectos.

    df_all = meteo.copy()
    df_all = df_all.sort_values(["estacion_id", "fecha"]).reset_index(
    ↪ drop=True)

    all_derived = []
    for est_id in sorted(df_all["estacion_id"].unique()):
        est_data = df_all[df_all["estacion_id"] == est_id].copy()
        est_data = est_data.sort_values("fecha").reset_index(drop=True)
        est_data = compute_derived_features(est_data)
        all_derived.append(est_data)

    df_all = pd.concat(all_derived, ignore_index=True)

```

```
    # Ahora filtramos filas excluidas (incendio == -1)
    df = df_all[df_all["incendio"] >= 0].copy()

    derived_cols = [
        "dias_sin_lluvia", "PP_acum_7d", "PP_acum_14d", "PP_acum_30d",
        "TA_AVG_3d", "TA_MAX_3d", "HR_MIN_3d", "amplitud_termica",
        "mes", "dia_del_ano",
    ]

    print(f"  Features calculadas sobre series continuas, luego
    ↪ filtradas")
    print(f"\n  Disponibilidad de features derivadas:")
    for col in derived_cols:
        nn = df[col].notna().sum()
        pct = nn / len(df) * 100
        print(f"    {col:<20} {nn:>8,} ({pct:.1f}%)")

    # --- STEP 5: Select final columns and save ---
    print(f"\n--- PASO 5: Seleccion de columnas y guardado ---")

    final_cols = (
        ["fecha", "estacion_id", "incendio"]
        + METEO_VARIABLES
        + derived_cols
    )

    result = df[final_cols].copy()

    # Final stats
    n_pos_final = (result["incendio"] == 1).sum()
    n_neg_final = (result["incendio"] == 0).sum()

    print(f"\n  Dataset final:")
    print(f"    Filas:     {len(result):,}")
    print(f"    Columnas:  {len(result.columns)}")
    print(f"    Positivos: {n_pos_final:,} ({n_pos_final/len(result)
    ↪ *100:.1f}%)")
    print(f"    Negativos: {n_neg_final:,} ({n_neg_final/len(result)
    ↪ *100:.1f}%)")
    print(f"    Ratio:     {n_neg_final/n_pos_final:.1f}:1")
    print(f"    Rango:     {result.fecha.min()} a {result.fecha.max()}")
    print(f"    Estaciones: {result.estacion_id.nunique()}")

    print(f"\n  Disponibilidad completa:")
    for col in result.columns:
        if col in ("fecha", "estacion_id", "incendio"):
```

```
            continue
        nn = result[col].notna().sum()
        pct = nn / len(result) * 100
        print(f"    {col:<20} {nn:>8,} ({pct:.1f}%)")

    # Save
    result.to_csv(OUTPUT_CSV, index=False)
    size_mb = os.path.getsize(OUTPUT_CSV) / (1024 * 1024)
    print(f"\n  Guardado: {OUTPUT_CSV}")
    print(f"  Tamano:   {size_mb:.2f} MB")


if __name__ == "__main__":
    main()
```

D.3 `entrenar_modelos.py`

Adestra o modelo de referencia (**Regresión Loxística**) sobre o dataset final. Usa unha división temporal (2005–2018 adestramento, 2019–2021 test) para simular un escenario real de produción. Avalía con varias métricas, optimiza o limiar de decisión mediante validación cruzada e garda as métricas para a comparativa posterior co Random Forest.

Comandos principais:

- `Pipeline([imputer, scaler, clf])`: cadea de tres pasos encadeados: imputación de valores faltantes (`SimpleImputer` pola mediana), normalización (`StandardScaler`, importante en modelos lineais) e o clasificador (`LogisticRegression`).
- `class_weight=balanced`: penaliza os erros sobre a clase minoritaria (incendios) case 20 veces máis que sobre a maioritaria, para que o modelo non a ignore.
- `TunedThresholdClassifierCV(model, scoring=f1, cv=5)`: axusta automaticamente o limiar de decisión mediante validación cruzada de 5 bloques, optimizando o F1-score.
- `average_precision_score(y_test, y_proba)`: métrica principal en datos desbalanceados: área baixo a curva precision-recall (PR-AUC).
- `precision_recall_curve`: calcula os puntos (recall, precision) para todos os limiares posibles; serve para o gráfico de avaliación.

```
#!/usr/bin/env python3
"""
```

```
Paso 12 — Entrenamiento y evaluacion de modelos de clasificacion.

Entrena modelos de forma incremental (baseline primero, luego mas complejos).
Cada modelo se evalua con F1, Recall, Precision, PR-AUC y umbral optimizado.

Uso: python3 src/entrenar_modelos.py
"""

import pandas as pd
import numpy as np
import json
import os
import sys
import warnings

warnings.filterwarnings("ignore", category=FutureWarning)

sys.path.insert(0, os.path.dirname(__file__))
from config import PROCESSED_DIR, METEO_VARIABLES

# --- PATHS ---
INPUT_CSV = os.path.join(PROCESSED_DIR, "dataset_modelo.csv")
OUTPUT_DIR = os.path.join(os.path.dirname(os.path.dirname(__file__)), "resultados")

# --- TEMPORAL SPLIT ---
# Train: 2005-2018 (14 years), Test: 2019-2021 (3 years)
# Temporal split avoids data leakage from future to past
TEST_YEAR_START = 2019

# --- FEATURES ---
DERIVED_FEATURES = [
    "dias_sin_lluvia", "PP_acum_7d", "PP_acum_14d", "PP_acum_30d",
    "TA_AVG_3d", "TA_MAX_3d", "HR_MIN_3d", "amplitud_termica",
    "mes", "dia_del_ano",
]
ALL_FEATURES = METEO_VARIABLES + DERIVED_FEATURES


def load_and_split(path):
    """Load dataset and split temporally into train/test."""
    df = pd.read_csv(path, parse_dates=["fecha"])

    train = df[df["fecha"].dt.year < TEST_YEAR_START]
```

```
    test = df[df["fecha"].dt.year >= TEST_YEAR_START]

    X_train = train[ALL_FEATURES]
    y_train = train["incendio"]
    X_test = test[ALL_FEATURES]
    y_test = test["incendio"]

    return X_train, y_train, X_test, y_test


def main():
    from sklearn.linear_model import LogisticRegression
    from sklearn.impute import SimpleImputer
    from sklearn.pipeline import Pipeline
    from sklearn.model_selection import TunedThresholdClassifierCV
    from sklearn.metrics import (
        f1_score, precision_score, recall_score,
        average_precision_score, confusion_matrix,
        precision_recall_curve,
    )
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    print("=" * 70)
    print("PASO 12.1 — LOGISTIC REGRESSION (BASELINE)")
    print("=" * 70)

    # --- LOAD & SPLIT ---
    X_train, y_train, X_test, y_test = load_and_split(INPUT_CSV)

    print(f"\n--- SPLIT TEMPORAL (train < {TEST_YEAR_START}, test >= {
    ↪ TEST_YEAR_START}) ---")
    for name, y in [("Train", y_train), ("Test", y_test)]:
        n_pos = (y == 1).sum()
        n_neg = (y == 0).sum()
        print(f"  {name}: {len(y):,} filas | "
              f"{n_pos:,} positivos ({n_pos/len(y)*100:.1f}%) | "
              f"{n_neg:,} negativos | "
              f"ratio {n_neg/n_pos:.1f}:1")

    train_nans = X_train.isna().sum().sum()
    test_nans = X_test.isna().sum().sum()
    print(f"\n  NaN en features: train={train_nans:,}, test={test_nans
    ↪ :,}")
    print(f"  Estrategia: SimpleImputer(strategy='median') en Pipeline")
```

```
90
91      # --- MODEL ---
92      print(f"\n--- MODELO: LOGISTIC REGRESSION ---")
93      print(f"  class_weight='balanced' (compensa desbalanceo
        ↪ automaticamente)")
94      print(f"  solver='lbfgs', max_iter=1000")
95
96      from sklearn.preprocessing import StandardScaler
97
98      model = Pipeline([
99          ("imputer", SimpleImputer(strategy="median")),
100         ("scaler", StandardScaler()),
101         ("clf", LogisticRegression(
102             class_weight="balanced",
103             max_iter=1000,
104             solver="lbfgs",
105             random_state=42,
106         )),
107     ])
108
109     model.fit(X_train, y_train)
110     print(f"  Entrenamiento completado.")
111
112     # --- EVALUATE BASE (threshold=0.5) ---
113     y_pred_base = model.predict(X_test)
114     y_proba = model.predict_proba(X_test)[:, 1]
115
116     f1_base = f1_score(y_test, y_pred_base)
117     recall_base = recall_score(y_test, y_pred_base)
118     precision_base = precision_score(y_test, y_pred_base, zero_division
        ↪ =0)
119     pr_auc = average_precision_score(y_test, y_proba)
120
121     print(f"\n--- RESULTADOS CON UMBRAL POR DEFECTO (0.5) ---")
122     print(f"  F1-score:  {f1_base:.4f}")
123     print(f"  Recall:    {recall_base:.4f}")
124     print(f"  Precision: {precision_base:.4f}")
125     print(f"  PR-AUC:    {pr_auc:.4f}")
126
127     cm = confusion_matrix(y_test, y_pred_base)
128     tn, fp, fn, tp = cm.ravel()
129     print(f"\n  Matriz de confusion:")
130     print(f"    Verdaderos negativos (dias sin incendio bien
        ↪ clasificados):  {tn:,}")
131     print(f"    Falsos positivos (falsas alarmas):
        ↪     {fp:,}")
```

```
    print(f"    Falsos negativos (incendios NO detectados):
    ↪     {fn:,}")
    print(f"    Verdaderos positivos (incendios detectados):
    ↪     {tp:,}")

    # --- THRESHOLD TUNING ---
    print(f"\n--- OPTIMIZACION DEL UMBRAL ---")
    print(f"  TunedThresholdClassifierCV(scoring='f1', cv=5)")
    tuned = TunedThresholdClassifierCV(model, scoring="f1", cv=5)
    tuned.fit(X_train, y_train)
    best_threshold = tuned.best_threshold_

    y_pred_tuned = (y_proba >= best_threshold).astype(int)
    f1_tuned = f1_score(y_test, y_pred_tuned)
    recall_tuned = recall_score(y_test, y_pred_tuned)
    precision_tuned = precision_score(y_test, y_pred_tuned,
    ↪ zero_division=0)

    print(f"\n--- RESULTADOS CON UMBRAL OPTIMIZADO ({best_threshold:.3f
    ↪ }) ---")
    print(f"  F1-score:  {f1_tuned:.4f}")
    print(f"  Recall:    {recall_tuned:.4f}")
    print(f"  Precision: {precision_tuned:.4f}")
    print(f"  PR-AUC:    {pr_auc:.4f} (no cambia, depende de
    ↪ probabilidades)")

    cm_tuned = confusion_matrix(y_test, y_pred_tuned)
    tn2, fp2, fn2, tp2 = cm_tuned.ravel()
    print(f"\n  Matriz de confusion:")
    print(f"    Verdaderos negativos:  {tn2:,}")
    print(f"    Falsos positivos:      {fp2:,}")
    print(f"    Falsos negativos:      {fn2:,}")
    print(f"    Verdaderos positivos:  {tp2:,}")

    # --- COEFFICIENTS (what LR learned) ---
    print(f"\n--- COEFICIENTES DEL MODELO (peso de cada variable) ---")
    coefs = model.named_steps["clf"].coef_[0]
    indices = np.argsort(np.abs(coefs))[::-1]
    for rank, idx in enumerate(indices, 1):
        sign = "+" if coefs[idx] > 0 else "-"
        print(f"  {rank:>2}. {ALL_FEATURES[idx]:<20} {sign} {abs(coefs[
    ↪ idx]):.4f}")

    # --- PLOT: PR curve ---
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    prec_curve, rec_curve, _ = precision_recall_curve(y_test, y_proba)
```

```
172
173     fig, ax = plt.subplots(figsize=(8, 6))
174     ax.plot(rec_curve, prec_curve, color="#4e79a7",
175             label=f"Logistic Regression (PR-AUC={pr_auc:.3f})")
176     ax.axhline(y=(y_test == 1).sum() / len(y_test), color="gray",
177                linestyle="--", label="Clasificador aleatorio")
178     ax.set_xlabel("Recall")
179     ax.set_ylabel("Precision")
180     ax.set_title("Curva Precision-Recall — Logistic Regression (baseline
    ↪ )")
181     ax.legend()
182     ax.grid(True, alpha=0.3)
183     ax.set_xlim([0, 1])
184     ax.set_ylim([0, 1])
185     fig.tight_layout()
186     fig.savefig(os.path.join(OUTPUT_DIR, "pr_curve_logistic_regression.
    ↪ png"), dpi=150)
187     plt.close(fig)
188     print(f"\n  Guardado: resultados/pr_curve_logistic_regression.png")
189
190     # --- SAVE METRICS for downstream comparison (entrenar_random_forest
    ↪ .py) ---
191     metrics = {
192         "model": "logistic_regression",
193         "test_year_start": TEST_YEAR_START,
194         "test_positive_rate": float((y_test == 1).sum() / len(y_test)),
195         "pr_auc": float(pr_auc),
196         "f1_base": float(f1_base),
197         "recall_base": float(recall_base),
198         "precision_base": float(precision_base),
199         "f1_tuned": float(f1_tuned),
200         "recall_tuned": float(recall_tuned),
201         "precision_tuned": float(precision_tuned),
202         "best_threshold": float(best_threshold),
203         "pr_curve": {
204             "recall": rec_curve.tolist(),
205             "precision": prec_curve.tolist(),
206         },
207     }
208     metrics_path = os.path.join(OUTPUT_DIR, "metrics_logistic_regression
    ↪ .json")
209     with open(metrics_path, "w", encoding="utf-8") as f:
210         json.dump(metrics, f, ensure_ascii=False, indent=2)
211     print(f"  Guardado: resultados/metrics_logistic_regression.json (
    ↪ para comparativa con RF)")
212
```

```
    # --- SUMMARY ---
    print(f"\n{'='*70}")
    print("RESUMEN — LOGISTIC REGRESSION (BASELINE)")
    print(f"{'='*70}")
    print(f"  PR-AUC:              {pr_auc:.4f}")
    print(f"  F1 (umbral 0.5):     {f1_base:.4f}")
    print(f"  F1 (umbral {best_threshold:.3f}):   {f1_tuned:.4f}")
    print(f"  Recall:              {recall_tuned:.4f}")
    print(f"  Precision:           {precision_tuned:.4f}")
    print(f"  Umbral optimo:       {best_threshold:.3f}")
    print(f"\n  Siguiente paso: entrenar Random Forest y comparar.")


if __name__ == "__main__":
    main()
```

D.4 `entrenar_random_forest.py`

Adestra o modelo principal (**Random Forest**) coa mesma división temporal e o mesmo protocolo de avaliación que a regresión loxística, para facer unha comparación xusta. Carga as métricas previas da regresión loxística dende o JSON gardado e xera os gráficos comparativos finais.

Comandos principais:

- `RandomForestClassifier(n_estimators=300, max_depth=15, min_samples`
  `class_weight=balanced, n_jobs=-1, random_state=42)`: configuración final do Random Forest. `n_jobs=-1` activa a paralelización en todos os núcleos da CPU; `random_state=42` fixa a semente para a reproducibilidade.
- `feature_importances_`: devolve o vector de importancia por impureza (MDI) de cada variable, base do ranking final.
- `TunedThresholdClassifierCV`: mesma estratexia de optimización do limiar que no baseline, para comparar en igualdade de condicións.
- **Carga das métricas LR previas**: o script le `metrics_logistic_regression.json` para calcular a mellora porcentual do Random Forest sobre o modelo de referencia en cada métrica.

```
#!/usr/bin/env python3
"""
Paso 14 — Entrenamiento de Random Forest.
```

```python

Entrena Random Forest sobre dataset_modelo.csv, compara con el baseline
de Logistic Regression, y genera graficos de feature importance.

Uso: python3 src/entrenar_random_forest.py
"""

import pandas as pd
import numpy as np
import json
import os
import sys
import warnings

warnings.filterwarnings("ignore", category=FutureWarning)

sys.path.insert(0, os.path.dirname(__file__))
from config import PROCESSED_DIR, METEO_VARIABLES

# --- PATHS ---
INPUT_CSV = os.path.join(PROCESSED_DIR, "dataset_modelo.csv")
OUTPUT_DIR = os.path.join(os.path.dirname(os.path.dirname(__file__)), "
    ↪ resultados")
LR_METRICS_JSON = os.path.join(OUTPUT_DIR, "metrics_logistic_regression.
    ↪ json")


def load_lr_metrics():
    """Load LR baseline metrics. Fail clearly if missing."""
    if not os.path.exists(LR_METRICS_JSON):
        raise FileNotFoundError(
            f"No se encontro {LR_METRICS_JSON}. "
            f"Ejecuta primero: python3 src/entrenar_modelos.py"
        )
    with open(LR_METRICS_JSON, "r", encoding="utf-8") as f:
        return json.load(f)

# --- TEMPORAL SPLIT ---
TEST_YEAR_START = 2019

# --- FEATURES ---
DERIVED_FEATURES = [
    "dias_sin_lluvia", "PP_acum_7d", "PP_acum_14d", "PP_acum_30d",
    "TA_AVG_3d", "TA_MAX_3d", "HR_MIN_3d", "amplitud_termica",
    "mes", "dia_del_ano",
]
```

```
ALL_FEATURES = METEO_VARIABLES + DERIVED_FEATURES


def load_and_split(path):
    """Load dataset and split temporally into train/test."""
    df = pd.read_csv(path, parse_dates=["fecha"])
    train = df[df["fecha"].dt.year < TEST_YEAR_START]
    test = df[df["fecha"].dt.year >= TEST_YEAR_START]
    return (
        train[ALL_FEATURES], train["incendio"],
        test[ALL_FEATURES], test["incendio"],
    )


def main():
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.impute import SimpleImputer
    from sklearn.pipeline import Pipeline
    from sklearn.model_selection import TunedThresholdClassifierCV
    from sklearn.metrics import (
        f1_score, precision_score, recall_score,
        average_precision_score, confusion_matrix,
        precision_recall_curve,
    )
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    print("=" * 70)
    print("PASO 14 — RANDOM FOREST")
    print("=" * 70)

    # --- LOAD & SPLIT ---
    X_train, y_train, X_test, y_test = load_and_split(INPUT_CSV)

    print(f"\n--- SPLIT TEMPORAL (train < {TEST_YEAR_START}, test >= {
    ↪ TEST_YEAR_START}) ---")
    for name, y in [("Train", y_train), ("Test", y_test)]:
        n_pos = (y == 1).sum()
        n_neg = (y == 0).sum()
        print(f"  {name}: {len(y):,} filas | "
              f"{n_pos:,} positivos ({n_pos/len(y)*100:.1f}%) | "
              f"{n_neg:,} negativos | "
              f"ratio {n_neg/n_pos:.1f}:1")

    # --- MODEL ---
```

```
    ratio = (y_train == 0).sum() / (y_train == 1).sum()
    print(f"\n--- MODELO: RANDOM FOREST ---")
    print(f"  n_estimators=300 (300 arboles)")
    print(f"  max_depth=15 (profundidad maxima por arbol)")
    print(f"  min_samples_leaf=10 (minimo 10 muestras por hoja)")
    print(f"  class_weight='balanced' (compensa desbalanceo {ratio:.0f}:1)")
    print(f"  Nota: no necesita StandardScaler (arboles son invariantes a escala)")

    model = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("clf", RandomForestClassifier(
            n_estimators=300,
            class_weight="balanced",
            max_depth=15,
            min_samples_leaf=10,
            random_state=42,
            n_jobs=-1,
        )),
    ])

    print(f"\n  Entrenando...")
    model.fit(X_train, y_train)
    print(f"  Entrenamiento completado.")

    # --- EVALUATE BASE (threshold=0.5) ---
    y_pred_base = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    f1_base = f1_score(y_test, y_pred_base)
    recall_base = recall_score(y_test, y_pred_base)
    precision_base = precision_score(y_test, y_pred_base, zero_division=0)
    pr_auc = average_precision_score(y_test, y_proba)

    print(f"\n--- RESULTADOS CON UMBRAL POR DEFECTO (0.5) ---")
    print(f"  F1-score:  {f1_base:.4f}")
    print(f"  Recall:    {recall_base:.4f}")
    print(f"  Precision: {precision_base:.4f}")
    print(f"  PR-AUC:    {pr_auc:.4f}")

    cm = confusion_matrix(y_test, y_pred_base)
    tn, fp, fn, tp = cm.ravel()
    print(f"\n  Matriz de confusion:")
    print(f"    Verdaderos negativos:  {tn:,}")
```

```
    print(f"    Falsos positivos:      {fp:,}")
    print(f"    Falsos negativos:      {fn:,}")
    print(f"    Verdaderos positivos:  {tp:,}")

    # --- THRESHOLD TUNING ---
    print(f"\n--- OPTIMIZACION DEL UMBRAL ---")
    print(f"  TunedThresholdClassifierCV(scoring='f1', cv=5)")
    tuned = TunedThresholdClassifierCV(model, scoring="f1", cv=5)
    tuned.fit(X_train, y_train)
    best_threshold = tuned.best_threshold_

    y_pred_tuned = (y_proba >= best_threshold).astype(int)
    f1_tuned = f1_score(y_test, y_pred_tuned)
    recall_tuned = recall_score(y_test, y_pred_tuned)
    precision_tuned = precision_score(y_test, y_pred_tuned,
    ↪ zero_division=0)

    print(f"\n--- RESULTADOS CON UMBRAL OPTIMIZADO ({best_threshold:.3f
    ↪ }) ---")
    print(f"  F1-score:  {f1_tuned:.4f}")
    print(f"  Recall:    {recall_tuned:.4f}")
    print(f"  Precision: {precision_tuned:.4f}")
    print(f"  PR-AUC:    {pr_auc:.4f}")

    cm_tuned = confusion_matrix(y_test, y_pred_tuned)
    tn2, fp2, fn2, tp2 = cm_tuned.ravel()
    print(f"\n  Matriz de confusion:")
    print(f"    Verdaderos negativos:  {tn2:,}")
    print(f"    Falsos positivos:      {fp2:,}")
    print(f"    Falsos negativos:      {fn2:,}")
    print(f"    Verdaderos positivos:  {tp2:,}")

    # --- FEATURE IMPORTANCE ---
    print(f"\n--- FEATURE IMPORTANCE (top 22) ---")
    importances = model.named_steps["clf"].feature_importances_
    indices = np.argsort(importances)[::-1]
    for rank, idx in enumerate(indices, 1):
        bar = "#" * int(importances[idx] * 100)
        print(f"  {rank:>2}. {ALL_FEATURES[idx]:<20} {importances[idx
    ↪ ]:.4f}  {bar}")

    # --- COMPARISON WITH BASELINE ---
    print(f"\n--- COMPARACION CON BASELINE (Logistic Regression) ---")
    print(f"  {'Metrica':<25} {'LogReg':>10} {'RandomForest':>14} {'
    ↪ Mejora':>10}")
    print(f"  {'-'*25} {'-'*10} {'-'*14} {'-'*10}")
```

```
178
179     # Carga las metricas reales de LR escritas por entrenar_modelos.py
180     lr_metrics = load_lr_metrics()
181     lr_pr_auc = lr_metrics["pr_auc"]
182     lr_f1 = lr_metrics["f1_tuned"]
183     lr_recall = lr_metrics["recall_tuned"]
184     lr_precision = lr_metrics["precision_tuned"]
185
186     for metric, lr_val, rf_val in [
187         ("PR-AUC", lr_pr_auc, pr_auc),
188         ("F1 (tuned)", lr_f1, f1_tuned),
189         ("Recall (tuned)", lr_recall, recall_tuned),
190         ("Precision (tuned)", lr_precision, precision_tuned),
191     ]:
192         if lr_val > 0:
193             mejora = (rf_val - lr_val) / lr_val * 100
194             print(f"  {metric:<25} {lr_val:>10.4f} {rf_val:>14.4f} {
    ↪ mejora:>+9.1f}%")
195         else:
196             print(f"  {metric:<25} {lr_val:>10.4f} {rf_val:>14.4f}")
197
198     # --- PLOTS ---
199     os.makedirs(OUTPUT_DIR, exist_ok=True)
200
201     # Plot 1: Feature importance
202     fig, ax = plt.subplots(figsize=(10, 7))
203     sorted_idx = np.argsort(importances)
204     ax.barh(range(len(sorted_idx)), importances[sorted_idx], color="#59
    ↪ a14f")
205     ax.set_yticks(range(len(sorted_idx)))
206     ax.set_yticklabels([ALL_FEATURES[i] for i in sorted_idx])
207     ax.set_xlabel("Importancia")
208     ax.set_title("Feature Importance — Random Forest")
209     fig.tight_layout()
210     fig.savefig(os.path.join(OUTPUT_DIR, "
    ↪ feature_importance_random_forest.png"), dpi=150)
211     plt.close(fig)
212     print(f"\n  Guardado: resultados/feature_importance_random_forest.
    ↪ png")
213
214     # Plot 2: PR curve REAL de RF vs PR curve REAL de LR (cargada del
    ↪ JSON)
215     prec_curve, rec_curve, _ = precision_recall_curve(y_test, y_proba)
216     lr_rec = lr_metrics["pr_curve"]["recall"]
217     lr_prec = lr_metrics["pr_curve"]["precision"]
218
```

```
    fig, ax = plt.subplots(figsize=(8, 6))
    ax.plot(rec_curve, prec_curve, color="#59a14f", linewidth=2,
            label=f"Random Forest (PR-AUC={pr_auc:.3f})")
    ax.plot(lr_rec, lr_prec, color="#4e79a7", linewidth=1.5, linestyle="
    ↪ --",
            label=f"Logistic Regression (PR-AUC={lr_pr_auc:.3f})")
    ax.axhline(y=(y_test == 1).sum() / len(y_test), color="gray",
               linestyle=":", alpha=0.7, label="Clasificador aleatorio")
    ax.set_xlabel("Recall")
    ax.set_ylabel("Precision")
    ax.set_title("Curva Precision-Recall — Random Forest vs Logistic
    ↪ Regression")
    ax.legend(loc="upper right")
    ax.grid(True, alpha=0.3)
    ax.set_xlim([0, 1])
    ax.set_ylim([0, 1])
    fig.tight_layout()
    fig.savefig(os.path.join(OUTPUT_DIR, "pr_curve_random_forest.png"),
    ↪ dpi=150)
    plt.close(fig)
    print(f"  Guardado: resultados/pr_curve_random_forest.png")

    # --- SUMMARY ---
    print(f"\n{'='*70}")
    print("RESUMEN — RANDOM FOREST")
    print(f"{'='*70}")
    print(f"  PR-AUC:              {pr_auc:.4f}")
    print(f"  F1 (umbral 0.5):     {f1_base:.4f}")
    print(f"  F1 (umbral {best_threshold:.3f}):   {f1_tuned:.4f}")
    print(f"  Recall:              {recall_tuned:.4f}")
    print(f"  Precision:           {precision_tuned:.4f}")
    print(f"  Umbral optimo:       {best_threshold:.3f}")
    print(f"\n  Siguiente paso: entrenar XGBoost y comparar los tres
    ↪ modelos.")


if __name__ == "__main__":
    main()
```

D.5 `exportar_modelo.py`

Exporta o Random Forest adestrado como un `Pipeline` serializado (*pickle*) para que a interface **LUMIA** o cargue directamente, sen necesidade de re-adestrar. Tamén exporta a importancia das variables e as métricas finais en JSON, para que LUMIA poida mostralas na

interface.

Comandos principais:

- `joblib.dump(pipeline, rf_pipeline.pkl)`: serializa o pipeline completo (imputer + Random Forest) nun único ficheiro binario que pode cargarse en milisegundos.
- `json.dump(feature_importance, ...)`: garda a importancia das variables en formato lexible, para que LUMIA poida mostrar o ranking sen necesidade do modelo.
- `json.dump(metrics, ...)`: exporta as métricas finais (PR-AUC, F1, limiar óptimo, etc.) para que LUMIA poida mostrar a fiabilidade do modelo na propia interface.

```
#!/usr/bin/env python3
"""
Exporta el Random Forest entrenado a un Pipeline serializado.

Uso: python3 src/exportar_modelo.py

Genera:
  interfaz/modelo/rf_pipeline.pkl       Pipeline (imputer + RF)
    ↪ serializado
  interfaz/modelo/feature_importance.json   Top 22 features con su peso
  interfaz/modelo/metrics.json          PR-AUC, threshold optimo, F1,
    ↪ etc.
"""

import json
import os
import sys
import warnings

import joblib
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    average_precision_score,
    f1_score,
    precision_score,
    recall_score,
)
from sklearn.model_selection import TunedThresholdClassifierCV
```

```
from sklearn.pipeline import Pipeline

warnings.filterwarnings("ignore", category=FutureWarning)

sys.path.insert(0, os.path.dirname(__file__))
from config import METEO_VARIABLES, PROCESSED_DIR

INPUT_CSV = os.path.join(PROCESSED_DIR, "dataset_modelo.csv")
OUTPUT_DIR = os.path.join(
    os.path.dirname(os.path.dirname(__file__)), "interfaz", "modelo"
)
TEST_YEAR_START = 2019

DERIVED_FEATURES = [
    "dias_sin_lluvia", "PP_acum_7d", "PP_acum_14d", "PP_acum_30d",
    "TA_AVG_3d", "TA_MAX_3d", "HR_MIN_3d", "amplitud_termica",
    "mes", "dia_del_ano",
]
ALL_FEATURES = METEO_VARIABLES + DERIVED_FEATURES


def main():
    os.makedirs(OUTPUT_DIR, exist_ok=True)

    print("=" * 70)
    print("EXPORTAR MODELO — Random Forest a pickle")
    print("=" * 70)

    df = pd.read_csv(INPUT_CSV, parse_dates=["fecha"])
    train = df[df["fecha"].dt.year < TEST_YEAR_START]
    test = df[df["fecha"].dt.year >= TEST_YEAR_START]
    X_train, y_train = train[ALL_FEATURES], train["incendio"]
    X_test, y_test = test[ALL_FEATURES], test["incendio"]
    print(f"\nTrain: {len(X_train):,} filas | Test: {len(X_test):,}
    ↪ filas")

    print("\nEntrenando Random Forest (300 árboles, max_depth=15)...")
    model = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("clf", RandomForestClassifier(
            n_estimators=300,
            class_weight="balanced",
            max_depth=15,
            min_samples_leaf=10,
            random_state=42,
            n_jobs=-1,
```

```
        )),
    ])
    model.fit(X_train, y_train)

    y_proba = model.predict_proba(X_test)[:, 1]
    pr_auc = average_precision_score(y_test, y_proba)

    print("Optimizando umbral con TunedThresholdClassifierCV...")
    tuned = TunedThresholdClassifierCV(model, scoring="f1", cv=5)
    tuned.fit(X_train, y_train)
    threshold = tuned.best_threshold_

    y_pred = (y_proba >= threshold).astype(int)
    f1 = f1_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, zero_division=0)

    print(f"\nMétricas finales:")
    print(f"  PR-AUC:    {pr_auc:.4f}")
    print(f"  Threshold: {threshold:.4f}")
    print(f"  F1:        {f1:.4f}")
    print(f"  Recall:    {recall:.4f}")
    print(f"  Precision: {precision:.4f}")

    # Guardar pickle del Pipeline + threshold + features
    bundle = {
        "pipeline": model,
        "threshold": float(threshold),
        "features": ALL_FEATURES,
        "pr_auc": float(pr_auc),
    }
    pkl_path = os.path.join(OUTPUT_DIR, "rf_pipeline.pkl")
    joblib.dump(bundle, pkl_path)
    size_mb = os.path.getsize(pkl_path) / (1024 * 1024)
    print(f"\nGuardado: {pkl_path}  ({size_mb:.1f} MB)")

    # Feature importance
    importances = model.named_steps["clf"].feature_importances_
    fi = sorted(
        [{"feature": ALL_FEATURES[i], "importance": float(importances[i
    ↪ ])}
         for i in range(len(ALL_FEATURES))],
        key=lambda x: -x["importance"],
    )
    fi_path = os.path.join(OUTPUT_DIR, "feature_importance.json")
    with open(fi_path, "w", encoding="utf-8") as f:
```

```
        json.dump(fi, f, ensure_ascii=False, indent=2)
    print(f"Guardado: {fi_path}")

    # Metrics summary
    metrics = {
        "model": "random_forest",
        "n_estimators": 300,
        "max_depth": 15,
        "min_samples_leaf": 10,
        "test_year_start": TEST_YEAR_START,
        "pr_auc": float(pr_auc),
        "threshold": float(threshold),
        "f1_tuned": float(f1),
        "recall_tuned": float(recall),
        "precision_tuned": float(precision),
    }
    m_path = os.path.join(OUTPUT_DIR, "metrics.json")
    with open(m_path, "w", encoding="utf-8") as f:
        json.dump(metrics, f, ensure_ascii=False, indent=2)
    print(f"Guardado: {m_path}")

    # Smoke test: cargar y predecir
    print("\nSmoke test: cargar pickle y predecir 1 muestra...")
    loaded = joblib.load(pkl_path)
    sample = X_test.iloc[[0]]
    p = loaded["pipeline"].predict_proba(sample)[0, 1]
    print(f"  Probabilidad muestra 0: {p:.4f}  (label real: {int(y_test.
    ↪ iloc[0])})")
    print(f"  Decision con threshold {loaded['threshold']:.3f}: "
          f"{'INCENDIO' if p >= loaded['threshold'] else 'no'}")
    print("\nOK.")


if __name__ == "__main__":
    main()
```

Bibliografía

[1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[2] Fundación Civio. Todos os incendios forestais. https://datos.civio.es/dataset/todos-los-incendios-forestales/, 2026. Dataset baseado na Estatística Xeral de Incendios Forestais (EGIF).

[3] Paulo Cortez and Aníbal Morais. A data mining approach to predict forest fires using meteorological data. In J. Neves, M. F. Santos, and J. Machado, editors, *Proceedings of the 13th Portuguese Conference on Artificial Intelligence (EPIA 2007)*, pages 512–523, Guimarães, Portugal, 2007.

[4] Fundación Civio. Galicia es la comunidad autónoma con más incendios y más superficie calcinada. https://civio.es/medio-ambiente/2024/09/05/galicia-es-la-comunidad-autonoma-con-mas-incendios-y-mas-superficie-calcinada/, 2024.

[5] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

[6] Duncan McElfresh, Sujay Khandagale, Jonathan Valverde, Vishak Prasad C, Benjamin Feuer, Chinmay Hegde, Ganesh Ramakrishnan, Micah Goldblum, and Colin White. When do neural nets outperform boosted trees on tabular data? *arXiv preprint arXiv:2305.02997*, 2023.

[7] MeteoGalicia. Datos diarios de estacións meteorolóxicas. api rest. https://servizos.meteogalicia.gal/mgrss/observacion/datosDiariosEstacionsMeteo.action, 2026.

[8] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Bru-

cher, M. Perrot, and É. Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[9] Ravid Shwartz-Ziv and Amitai Armon. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90, 2022.

[10] John P. Snyder. *Map Projections—A Working Manual.* Professional Paper 1395. U.S. Geological Survey, Washington, D.C., 1987.

[11] Somos Comarca. Valdeorras, epicentro del peor verano de incendios en galicia: más de 90.000 hectáreas arrasadas en ourense. https://www.somoscomarca.es/articulo/actualidad/valdeorras-epicentro-peor-verano-incendios-galicia-mas-90000-hectareas-arrasadas-ourense/20250901102007202789.html, 2025. 1 de setembro de 2025.

[12] Xunta de Galicia — Medio Rural. Índice de risco diario de incendio (irdi). https://mediorural.xunta.gal/es/temas/defensa-monte/irdi, 2026.

[13] Daochen Zha, Sai Praneeth Bhat, Kwei-Herng Lai, Fan Yang, and Xia Hu. Balancing the scales: A comprehensive study on class imbalance. *arXiv preprint arXiv:2409.19751*, 2024.